

C0 Act Aggregator — Method Comparison

Aggregating C1 measure names into act-level canonical clusters

Esteban Degetau
World Bank
estebandegetau@gmail.com

Agustín Samano
World Bank
asamanopenaloza@worldbank.org

June 30, 2026

Framing

C1 v0.7.0 extracts a `measure_name` string per chunk $\times$ measure, but the same fiscal act surfaces under many different forms (“ERTA”, “Economic Recovery Tax Act of 1981”, “Economic Recovery Tax Act (ERTA)”, “the 1981 tax cut”). Downstream consumers — C2b motivation, deployment summaries, and any cross-country comparison — need a stable canonical mapping at the **act** level. The job of a “C0” Act Aggregator is to produce that mapping.

The Malaysia EN/BM consistency notebook (`malay_consistency.qmd`) showed that JW single-linkage at threshold 0.15 fragments acts (especially BM-side, where “Subsidi Bahan Api” + “Subsidi Gula” split off from a single budget envelope), and that this fragmentation propagates downstream: C2b reads asymmetric evidence on the two language sides and apparent disagreement is partly clustering noise. So before designing a C0 codebook we need empirical evidence on which aggregation strategy holds up.

This notebook compares five methods on a label-anchored evaluation set (US C1 outputs joined to Romer & Romer’s 40 gold-standard acts) and an out-of-distribution stress test (Malaysia EN/BM paired pairs).

Scope. Corpus-wide per country (one canonical list across all years of a corpus), not per-document and not cross-country. The C1 deployment runs per-country; C0 will too.

Asymmetric failure cost. Over-merging (collapse two real acts into one cluster) corrupts C2b inputs — bad. Under-merging (one act stays split across two clusters) just produces parallel-but-correct classifications — recoverable downstream. The headline pick should weight precision over recall.

From asymmetric cost to coverage + spurious rate. The operational form of the asymmetric-cost argument depends on what gold the eval scores against. Earlier drafts of this notebook used pairwise Precision/Recall against `c0_eval_gold_pairs` (the chunk-tier-tagged pool described below), with the reading that pairwise precision was costly to lose and pairwise recall recov-

erable. The current draft scores against Romer & Romer’s 49-act list in `us_shocks.csv` — independent of any pipeline output. The asymmetry translates cleanly: **low spurious rate** (few clusters that match no RR act) takes the place of high precision; **high coverage** (many RR acts hit by some cluster) takes the place of high recall; **low fragmentation** (few clusters per matched RR act) is the new precision-flavoured tiebreak. §“Why we pivoted away from pairwise P/R” below documents the structural ceiling that forced the change.

Gold partition caveat. The new framework scores against R&R’s act list as if it were ground truth. If R&R’s partition splits something a domain expert would have merged (or vice versa), the metric labels inherit that. The chunk-tier leakage caveat that applied to the old pairwise eval no longer applies (the RR-aligned framework bypasses chunk-tier tagging entirely), but the act-list correctness assumption is the same.

What “win” means. High coverage with low spurious rate, low fragmentation index, and high year alignment (most name-matched clusters also satisfy a ± 2 -year window around the act’s signing year). Metrics reported under two complementary name gates side by side — keyword containment (precision-friendly, immune to name-rewriting) and $\text{JW-min} \leq 0.30$ (continuous, sensitive to abbreviations). The gap between the two surfaces methods that emit RR-style canonical names without doing more aggregation work. Bootstrap 95% CIs with $n_{\text{boot}} = 1000$ in the pipeline; the act-indexed metrics resample the 49 RR acts, while spurious rate resamples the emitted clusters (its natural sampling unit) — see §“Bootstrap CI”.

Descriptive baseline

Table 1 sizes the canonicalization problem at the chunk-measure level. Its five rows describe the C1 v0.7.0 output pool feeding C0:

- **Total FISCAL_MEASURE rows** counts chunk $\times$ measure entries C1 flagged as FISCAL_MEASURE. Because v0.7.0 returns a `measures[]` array per chunk, one chunk can contribute multiple rows.
- **Distinct measure_name strings (any rank)** counts unique surface forms across the full pool, including secondary measures from chunks whose primary measure was something else. This is the maximum pool size C0 would face if it had to canonicalize every name C1 ever emits.
- **Distinct measure_name strings (rank == 1)** counts only the primary surface form per chunk. The codebook contract (`c1_measure_id.yml`) requires the first array element to be the most prominent measure discussed, so `rank == 1` is C1’s primary pick per chunk. This is the realistic pool size for a primary-measure-only deployment, which matches the current C2b assumption.
- **Years spanned** is the inclusive range of document years contributing chunks.
- **Distinct doc_id** is the number of source documents represented.

The gap between rows 2 and 3 quantifies how much of the surface-form pool comes from secondary mentions. A small gap means rank-1 carries most of the variance and C0 can be evaluated on the rank-1 subset without much loss. A

large gap means non-primary measures contribute substantial unique forms, and C0’s effective scope (primary-only vs all) becomes a load-bearing methodological choice. The eval set in §3 is built on rank == 1 only.

Table 1: **Measure-name pool.** Pool summary from c1_s2_results.

Metric	Value
Total FISCAL_MEASURE rows	2941
Distinct measure_name strings (any rank)	1976
Distinct measure_name strings (rank == 1)	294
Years spanned	1946-2007
Distinct doc_id	129

Surface-form variance per gold act — the magnitude of the canonical-ization problem. For each gold act_name with at least one chunk in `c0_eval_gold_pairs`, count the distinct C1 surface forms (rank=1 only) that map to it.

Figure 1: **Surface-form variance per act.** Distinct C1 rank-1 surface forms per gold act (top 25 most fragmented).

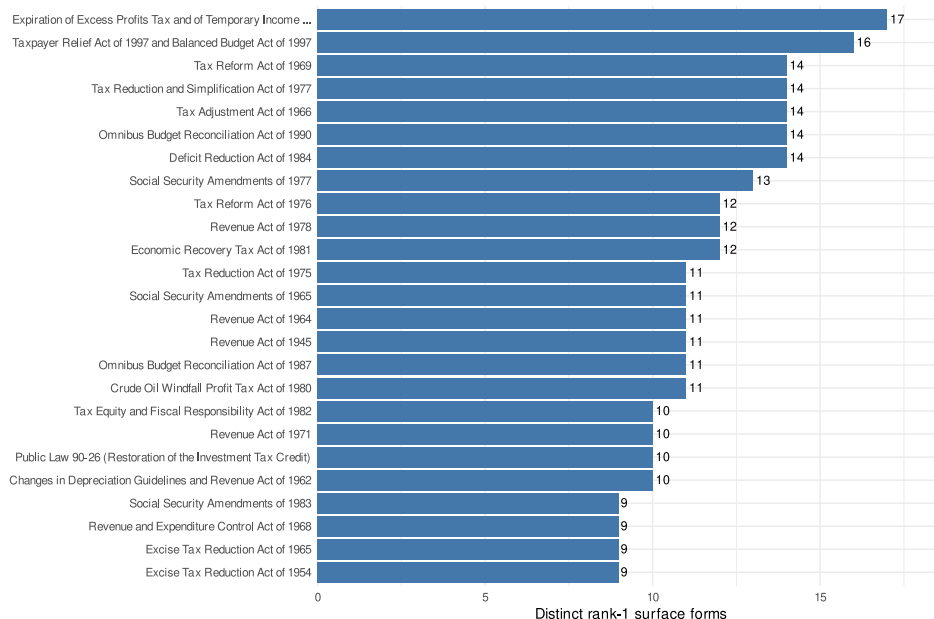


Table 2 drills into the same data as Figure 1, using the full any-rank measure pool (rather than the rank-1 subset used by the figure). Gold acts appear as group headers most-fragmented first, annotated with the total number of distinct C1 surface forms that ever appeared in chunks tagged with that act at any rank. Under each group, the top five most-frequently-emitted surface forms with the count of chunk-rows in which C1 emitted them. The shape of the within-group distribution is what C0 has to recover: a dominant first form with a long tail rewards conservative merging because the dominant form is already a service-

able canonical; a flat distribution needs more aggressive merging to collapse parallel forms.

Table 2: **Surface-form detail per act.** Top 5 C1 surface forms per gold act (any rank). Group headers list the total number of distinct surface forms for the act; acts with fewer than five forms show all of them.

Surface form	Appearances
Social Security Amendments of 1977 (946 surface forms)	
Revenue Act of 1964	20
Economic Recovery Tax Act of 1981	18
Social Security Amendments of 1977	18
Tax Reform Act of 1976	18
Social Security Amendments of 1983	17
Revenue Act of 1971 (922 surface forms)	
Revenue Act of 1971	22
Revenue Act of 1950	18
Revenue Act of 1964	16
Social Security Amendments of 1983	15
Tax Reform Act of 1986	14
Revenue Act of 1964 (883 surface forms)	
Revenue Act of 1964	24
Economic Recovery Tax Act of 1981	13
Social Security Amendments of 1983	13
Revenue Act of 1950	11
Excise Tax Reduction Act of 1965	10
Omnibus Budget Reconciliation Act of 1993 (862 surface forms)	
Revenue Act of 1964	20
Revenue Act of 1950	15
Economic Recovery Tax Act of 1981	14
Social Security Amendments of 1983	13
Tax Equity and Fiscal Responsibility Act of 1982	12
Deficit Reduction Act of 1984 (835 surface forms)	
Economic Recovery Tax Act of 1981	21
Social Security Amendments of 1983	21
Deficit Reduction Act of 1984	18
Highway Revenue Act of 1982	17
Tax Equity and Fiscal Responsibility Act of 1982	17
Taxpayer Relief Act of 1997 and Balanced Budget Act of 1997 (816 surface forms)	
Revenue Act of 1964	19
Revenue Act of 1964 (5 surface forms)	
Revenue Act of 1964 (5 surface forms)	

Table 3: **Fragmentation distribution.** Distribution of surface-form fragmentation across the 39 gold acts.

Metric	Value
Acts evaluated	39.00
Median surface forms / act	10.00
Mean surface forms / act	9.54
Max surface forms / act	17.00
Acts with ≥ 5 forms	34.00
Acts with exactly 1 form	1.00

The headline: even with R&R’s well-known and well-named acts, the median act surfaces under multiple forms; the tail is heavy. A C0 that returns one cluster per surface form (i.e., no merging) is useless; a C0 that merges everything is also useless. The methods below try to land somewhere useful.

Within-document name variance: 1980 cross-section

Figure 2 sizes the data reduction happening upstream of the heatmaps below. Stage 1 counts every chunk C1 reviewed in each 1980 document (the input to FISCAL_MEASURE classification), split into the “relevant” chunks where C1 emitted at least one measure and the irrelevant chunks where it did not. Stage 2 keeps only the relevant subset — the same blue block from stage 1 redrawn on its own to make the funnel mouth visible. Stage 3 unfolds the relevant chunks into individual measure occurrences (one row per chunk $\times$ `measure_rank` pair), arranged by rank with the rank-1 layer marked in red. The shape of the funnel is the headline: Budget and Treasury chunks emit ≈ 5 -10 measures each, so the rank-1 red band is a small slice of stage 3 — and that red slice is the pool the JW and embedding heatmaps below operate on. Rank ≥ 2 layers (grey) are out of scope for the heatmaps but are not noise; they record secondary measures C1 surfaced in the same chunk.

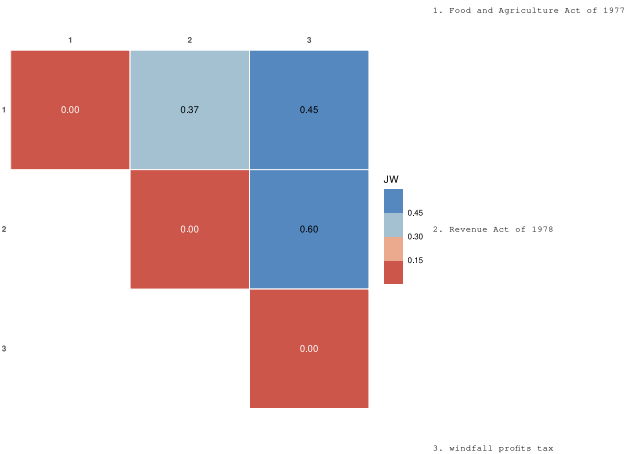
Figure 2: **Per-document funnel.** From C1 input to the rank-1 layer the heatmaps below operate on. Stage 1: chunks C1 reviewed, split into relevant (FISCAL_MEASURE emitted) and irrelevant. Stage 2: the relevant subset alone. Stage 3: all FISCAL_MEASURE occurrences (one per chunk $\times$ `measure_rank` pair) stacked by rank, with rank-1 marked in red. Stages 1-2 are counted in chunks; stage 3 in occurrences (so a single relevant chunk producing five measures contributes 1 to stage 2 and 5 to stage 3). The y-axis is free per facet to keep ERP-1980’s small counts legible alongside Treasury’s larger ones.



Figure 3 zooms into the rank-1 layer of Figure 2: it shows the pairwise Jaro-Winkler distance matrix among those rank-1 C1 measure names within each document (mirroring the Methods 2 + 3 pool used in Figure 4, so the two heatmaps carry the same acts). The heatmap pool is the within-doc surface forms intersected with the corpus-wide rank-1 pool (`rownames(c0_us_embeddings)`) — slightly larger than stage 3’s red layer because it includes a few names that are at rank ≥ 2 in *this* document but were rank-1 in some other document, while excluding rank-1 occurrences whose surface form is shared with another (so the same name shown once). Cells in the bottom band ($JW < 0.15$) would merge under single-linkage at the production threshold. Cells in the middle band ($0.15-0.30$) are near-misses — same measure with verbose-vs-bare phrasing, swapped word order, or abbreviation expansion. Cells in the top band (≥ 0.30) are genuinely distinct measures. Read each heatmap together with its numbered measure list (printed to its right). Off-diagonal cells just above the 0.15 line that link semantically-identical names are the failure cases that drive over-fragmentation; cells far above 0.30 linking surface forms that *are* the same act are the gap JW cannot close on its own.

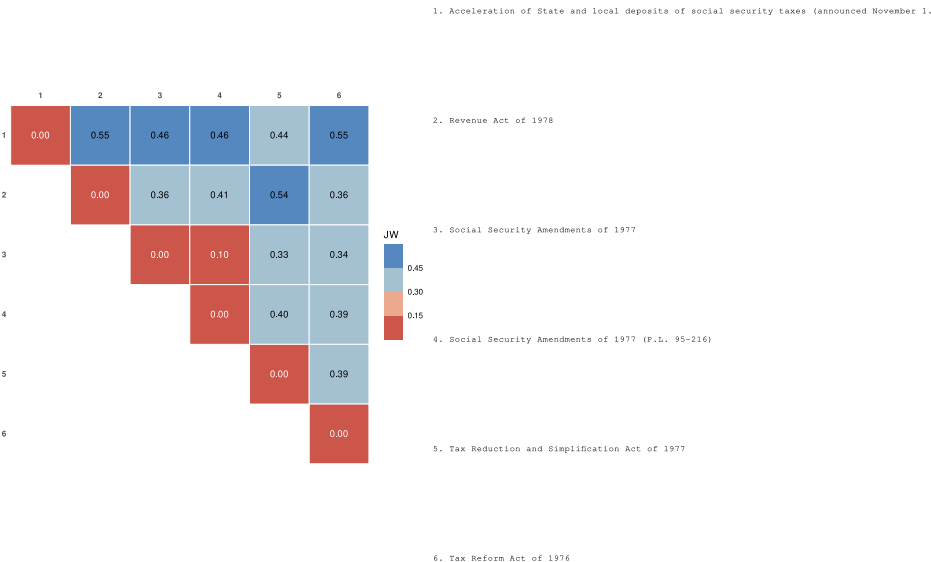
(a) Economic Report of the President

ERP-1980 — 3 rank-1 C1 measure names (2 non-rank-1 omitted)



(b) Budget

BUDGET-1980 — 6 rank-1 C1 measure names (23 non-rank-1 omitted)



(c) Treasury Annual Report

AR_TREASURY-1980 — 10 rank-1 C1 measure names (16 non-rank-1 omitted)

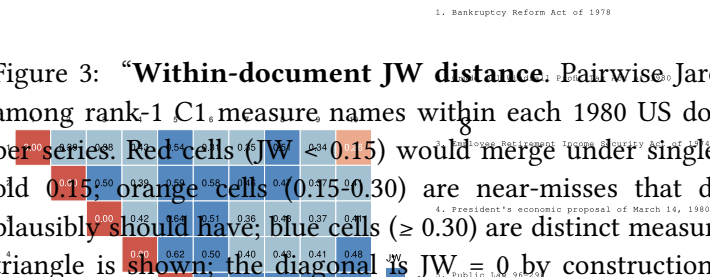


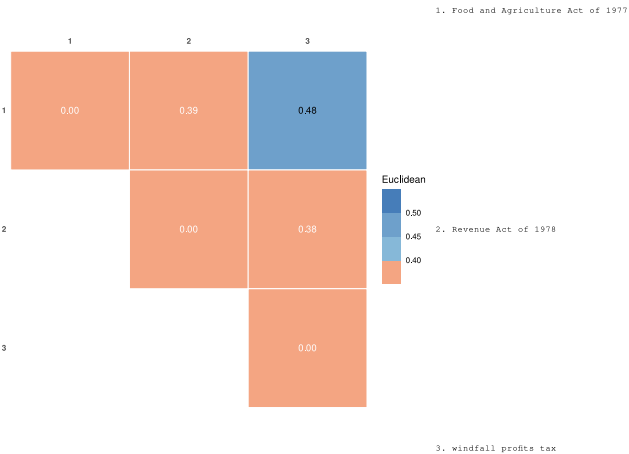
Figure 3: “**Within-document JW distance.** Pairwise Jaro-Winkler distance among rank-1 C1 measure names within each 1980 US document, one panel per series. Red cells ($JW < 0.15$) would merge under single-linkage at threshold 0.15; orange cells (0.15-0.30) are near-misses that did not merge but plausibly should have; blue cells (≥ 0.30) are distinct measures. Only the upper triangle is shown; the diagonal is $JW = 0$ by construction. The pool is each

Figure 3 visualised what JW sees within each of the three 1980 US documents. Figure 4 is the same structure on the same documents and the same measure-name pool, but with pairwise *embedding* distance in place of JW. Distances are Euclidean on the L2-normalised E5 vectors `c0_us_embeddings`; because the embeddings are unit-norm, Euclidean distance is a monotonic transform of cosine distance ($d_{\text{euc}} = \sqrt{2 \cdot d_{\text{cos}}}$), so the figure mirrors what Methods 2 + 3 actually cluster on.

The colour bands are tighter than the JW figure’s because E5 packs everything into a narrow band of the unit sphere: corpus-wide pairwise Euclidean distances sit at median 0.45 with IQR 0.42-0.48. Bands at 0.40 / 0.45 / 0.50 therefore approximate “below corpus IQR” / “within IQR” / “above IQR” / “far above IQR” — a relative-rather-than-absolute reading, in keeping with the verdict text above.

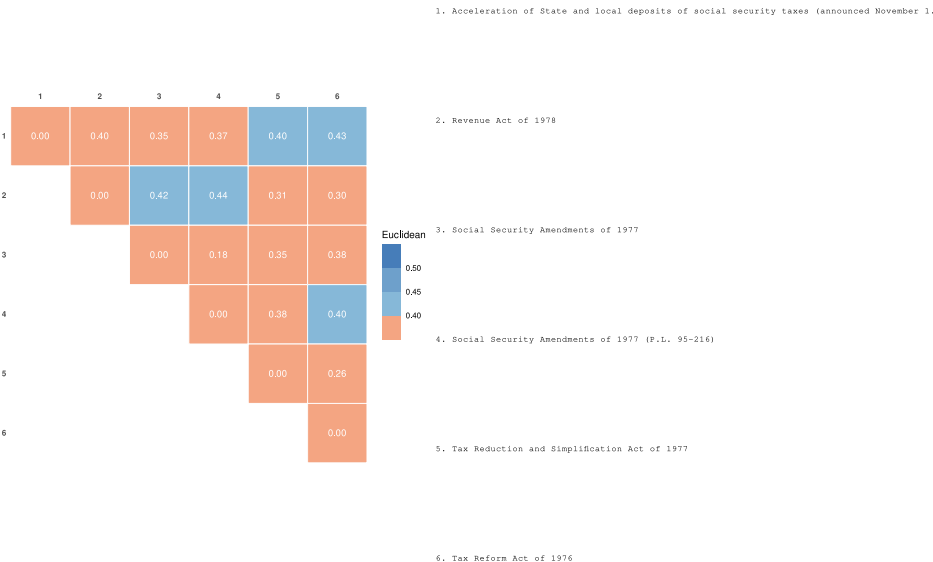
(a) Economic Report of the President

ERP-1980 — 3 embedded C1 measure names (2 without embedding omitted)



(b) Budget

BUDGET-1980 — 6 embedded C1 measure names (23 without embedding omitted)



(c) Treasury Annual Report

AR_TREASURY-1980 — 10 embedded C1 measure names (16 without embedding omitted)

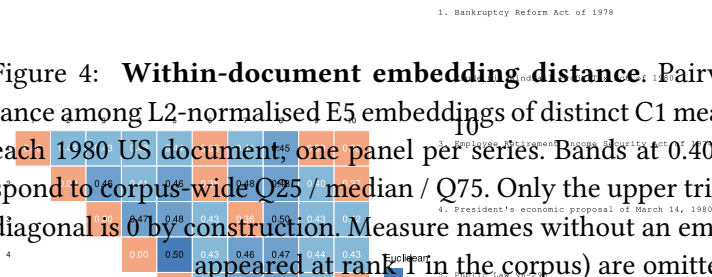


Figure 4: **Within-document embedding distance.** Pairwise Euclidean distance among L2-normalised E5 embeddings of distinct C1 measure names within each 1980 US document; one panel per series. Bands at 0.40 / 0.45 / 0.50 correspond to corpus-wide Q25 / median / Q75. Only the upper triangle is shown; the diagonal is 0 by construction. Measure names without an embedding (i.e., never appeared at rank 1 in the corpus) are omitted.

Reading Figure 4 against Figure 3, an immediate conclusion is that imposing the Rank-1 filter as in Figure 4 does not imply losing important fiscal changes. Indeed, doing so, refines the surfaced names and are more in tune with the acts Romer and Romer (n.d.) identify. Importantly, there is little within-document-within act name variance and it could be recuperated both with JW distance and with embeddings.

Tier-tagged eval set (descriptive)

This section documents `c0_eval_gold_pairs`, the chunk-tier-tagged pool used by the embedding-geometry probes in §“Embedding-geometry probe (F16 vs FP32)” and §“Eval-set noise probe”. It is **not** the headline gold for method comparison — that role belongs to R&R’s 49-act list, evaluated under the framework defined in §“Evaluation against RR’s act list” below. The structural reason for the split is in §“Why we pivoted away from pairwise P/R”, which follows immediately.

The eval set projects the chunk-tier system through C1’s LLM output along the chain `aligned_data` → `c1_chunk_data` → `c1_s2_results` → `c0_eval_gold_pairs`:

1. `c1_chunk_data` (built by `R/identify_chunk_tiers.R`) emits `(chunk_id, doc_id, act_name, tier)` rows by string-matching gold passages and act-name subcomponents against chunk text. Tier 1 marks chunks containing a verbatim gold passage; Tier 2 marks chunks matching an act name via sub-component decomposition or co-occurrence rules. Chunks matching multiple acts within the same tier are row-replicated, one row per `(chunk × act)`. The Tier 1 / Tier 2 exclusion is chunk-level, not `(chunk, act)`-level: any chunk that is Tier 1 for *some* act is barred from Tier 2 consideration for *every* act, so Tier 2 evidence on mixed-tier multi-act chunks is dropped at the source.
2. `c1_s2_results` runs the C1 LLM on these tagged chunks via leave-one-act-out cross-validation; each `(chunk × gold act)` pair is its own fold with its own few-shot set. The LLM’s emitted `measures[]` array is exploded into one row per `(chunk × gold act × measure_rank)`, so a chunk tagged with two gold acts is classified twice and may carry different `measure_name` strings under each gold tag.
3. `build_c0_eval_gold_pairs()` (in `R/c0_aggregator.R`) keeps only rows with `measure_rank == 1`, `pred_label == FISCAL_MEASURE`, and `tier ∈ {1, 2}`; renames `act_name` → `gold_act_name`; and flags surface forms paired with >1 distinct gold act as `ambiguous` for the evaluator to majority-vote.

Two consequences carry into every downstream metric. (i) The rank-1 filter discards non-primary measures the LLM emitted, so a chunk discussing N gold acts contributes at most N rows with only the rank-1 surface form for each. (ii) Chunks the LLM mislabeled `NOT_FISCAL_MEASURE` drop out entirely, silently truncating multi-act representation whenever the LLM’s per-fold output is inconsistent.

Table 4: **Eval-set composition.** Rows count (`measure_name`, `gold_act_name`, `chunk_id`) tuples; distinct surface forms count unique `measure_name` strings — the population the clusterer canonicalizes. The two indented rows partition the Tier 1 + Tier 2 pool by whether each surface form maps to one (unambiguous) or more (ambiguous) gold acts. Percentages are shares of the Tier 1 + Tier 2 totals.

Slice	Chunk × gold-act rows	Distinct surface forms	Gold acts covered
Tier 1 (verbatim passage match)	60	54	30
Tier 1 + Tier 2 (full eval pool)	592	267	39
↳ unambiguous names (1 gold act per name)	237 (40%)	206 (77%)	—
↳ ambiguous names (≥2 gold acts per name)	355 (60%)	61 (23%)	—

On ambiguity. Roughly 60% of the Tier 1+2 rows have a `measure_name` that’s been paired with more than one `gold_act_name` across chunks. Most cases trace to Tier 2’s permissive matching: a chunk may be tagged with act X because act X is mentioned somewhere, but the most-prominent measure C1 extracts is a different act. The evaluator handles this by majority-voting per `measure_name` within each bootstrap resample (so the contingency table operates on unique names). Tier 1 is shown as the cleaner anchor; Tier 1+2 is shown for statistical power.

Gold-set leakage caveat. The Tier 1/Tier 2 tags themselves were assigned by fuzzy-matching the gold `act_name` against the chunk text, so methods that exploit surface similarity (especially JW) will look artificially strong on this evaluation. Treat the per-act recall column as a complement — methods that nail F1 but miss specific acts may be exploiting the leakage.

Why we pivoted away from pairwise P/R

`c0_eval_gold_pairs` has a structural ceiling on pairwise recall that comes from how the pool is constructed, not from any clusterer’s performance. `build_c0_eval_gold_pairs()` assigns *every* rank-1 measure C1 extracted in a chunk to *every* gold act that chunk was tier-tagged with — including chunks where C1’s primary measure was a different act. The act `gold_act_name` becomes a bag of co-occurring rank-1 measures rather than a list of surface variants of one act. Concretely, “Tax Reform Act of 1969” picks up members {ERISA, Asset Depreciation Range (ADR), H.R. 10947, tax bill passed in December} — distinct policy items, not paraphrases. No string-embedding clusterer can or should merge those.

Even on the Tier 1 slice (verbatim-passage tag, supposedly the clean anchor), the same pattern holds. “Deficit Reduction Act of 1984” Tier-1 members include {Economic Recovery Tax Act, Job Training Partnership Act, tax proposals (marginal income tax rate reductions and depreciation reform)}. The Tier 1 filter constrains where C1 emitted its measure, not what measure it emitted.

The §UMAP investigation in the 2026-05-28 commit cycle confirmed this empirically: 16 of 35 eligible Tier 1 acts have top-1 NN same-act rate = 0 on F16 raw embeddings (20/35 under the UMAP precision-friendly winner); across the 162-cell UMAP grid, `cor(% acts reconstructed intact, F1) = +0.82` while `cor(separation, pairwise_recall) = -0.07` — pairwise F1 is essentially a one-variable story about how often a gold act survives HDBSCAN as a single cluster, and that variable is capped by gold-pool construction upstream of the embedding. No UMAP variant strictly dominated the no-UMAP baseline on the original pairwise eval because both hit the same ceiling. See §“Reading the UMAP results” for the full diagnostic.

The pivot. Method comparison now scores against R&R’s authoritative 49-act list in `us_shocks.csv`, which is independent of any pipeline output. The pairwise-P/R evaluation against `c0_eval_gold_pairs` is preserved for the embedding-geometry probes (where median within/between separation is the metric, not a partition recall) and the eval-set noise probe (where the temporal mismatch is the diagnostic) — but no longer drives method selection.

Evaluation against RR’s act list

Reference: what `us_shocks.csv` provides

`c0_us_rr_acts` collapses `us_shocks` (one row per act × quarter × measure_type) to one row per distinct act with `year_signed = year(Date signed)`. 49 acts spanning 1945–2007. The first few rows:

Table 5: **RR canonical-act list.** First five rows of `c0_us_rr_acts`, sorted by `year_signed`.

act_id	act_name	year_signed
1	Revenue Act of 1945	1945
2	Social Security Amendments of 1947	1947
3	Revenue Act of 1948	1948
4	Revenue Act of 1950	1950
5	Social Security Amendments of 1950	1950

This list is the metric’s reference: a predicted cluster is “good” iff some member surface form (and the cluster’s representative year) matches an entry in this list under the gates below.

Matching gates

Two complementary name gates, evaluated side by side in every metric table that follows. Both combined with a year window.

Keyword containment (precision-friendly). For each RR act, decompose the act name via `generate_subcomponents()` (the function `R/identify_chunk_tiers.R` already uses for Tier 2 chunk tagging — same decomposition, no new code path) into a set of searchable terms: the full canonical name, “and”-splits, parentheticals, embedded “{X} Act of {YYYY}” patterns, Public Law numbers, action+subject phrases. The shared `BROAD_EXCLUSIONS` list strips overly generic fragments. A cluster keyword-matches an RR act iff *any* of its surface forms (after `squish_for_matching()` normalisation) contains *any* of the act’s subcomponent terms. Property: immune to name-rewriting — a method that re-emits the canonical act name as cluster representative gets no extra credit because the test runs against the cluster’s *members*, not its canonical.

JW-min (continuous, abbreviation-sensitive). For each (cluster, RR act) pair, compute `min` over cluster members of `stringdist::stringdist(member, rr_act_name, method = "jw")` on lowercased strings. Threshold ≤ 0.30 . Property: catches near-canonical matches that keyword-containment misses (e.g. typographic variants); also catches *false* matches across acts whose names share rare tokens. The JW gate is the relaxed one — it rewards methods that emit RR-style canonical names because those land at JW = 0 against the act.

The gap between coverage under keyword and coverage under JW is itself diagnostic: a method whose keyword and JW coverage agree is recovering the canonical forms via aggregation; a method with much higher JW coverage than keyword coverage is mostly re-emitting canonical names without aggregating much.

Year window

Cluster representative year = median of years extracted from members’ surface forms via `extract_year_from_string()` (regex `\b(19|20)\d{2}\b`), falling back to the mode of `doc_year` from `c0_us_measure_pool` when no member name carries an extractable year. Year gate: $|\text{cluster_year} - \text{year_signed}| \leq 2$. Width ± 2 chosen to (i) align with the Method 3 year-blocking variants so methods that respect the year prior get credit, and (ii) absorb retrospective references whose mode `doc_year` lags the signing year by 1–2.

Joint match gate per name gate: name match AND year match. A cluster that name-matches an act but is 30 years off does *not* match.

Headline metrics

Under each name gate, four numbers per variant. The arrow in each header gives the direction of improvement — **↑ greater is better**, **↓ lower is better**:

- **Coverage** (↑ greater is better) = $|\{\text{RR acts with } \geq 1 \text{ matching cluster}\}| / 49$. Recall-flavoured: a higher value means more of R&R’s 49 acts were recovered by some cluster.

- **Spurious rate** ($\downarrow$ lower is better) = $|\{\text{clusters with no RR-act match}\}| / n_clusters$. Precision-flavoured; this is the metric the asymmetric-cost framing weights most. A lower value means fewer junk clusters that match nothing in R&R’s list.
- **Fragmentation index** ($\downarrow$ lower is better) = mean over matched RR acts of ($\#$ distinct clusters claiming that act). ≥ 1 by construction. Equals 1 (the ideal) when each matched act has exactly one cluster; grows when methods scatter one act across several clusters.
- **Year alignment** ($\uparrow$ greater is better) = of name-matched pairs (name gate only, ignoring the year gate), share where the year gate also passes. Tests whether name and year signals agree — low values under the JW gate indicate the JW threshold is producing cross-decade false positives.

Coverage and spurious rate are the headline pair. Fragmentation is the precision-flavoured tiebreak between methods at similar coverage. Year alignment is mostly diagnostic.

The shortlist tables in §Synthesis (e.g. Table 23, Table 25) carry three additional count columns that anchor these rates in absolute numbers; all three are descriptive context for the rates above, not gates with a preferred direction:

- **n clusters** = total clusters the variant emitted on the corpus pool. Read it as the denominator behind the spurious rate and as a coarse aggregation-effort signal: a value near the surface-form pool size means the method barely merged anything (near-singletons), while a much smaller value means aggressive merging. Not “greater/lower is better” on its own — it’s only meaningful read against coverage and spurious rate (few clusters with high coverage is good; few clusters with low coverage means over-merging).
- **Clusters matched** = $|\{\text{clusters matching } \geq 1 \text{ RR act under the joint gate}\}| = n_clusters \times (1 - \text{spurious rate})$. The count of “useful” clusters; the spurious rate is its complement expressed as a share.
- **RR acts matched** = $|\{\text{of the 49 RR acts hit by } \geq 1 \text{ cluster}\}| = \text{coverage} \times 49$. The numerator behind coverage, in act counts rather than a share.

Comparing **Clusters matched** against **RR acts matched** reads off fragmentation directly: when the two are equal each matched act has exactly one cluster (fragmentation = 1); when Clusters matched exceeds RR acts matched, the surplus clusters are the fragmentation.

Bootstrap CI

The four metrics are bootstrapped over **two different sampling units**, because they are proportions over different populations. `n_boot = 1000` in the pipeline (the smoke test uses 100 for speed); both bootstraps take the 0.025 and 0.975 percentiles.

Coverage, fragmentation, and year alignment — resample the 49 RR acts.

These are act-indexed, so resampling acts with replacement perturbs *which* acts the method is held accountable for — the noise the CI should capture (acts vary

in difficulty for any given clusterer). This bootstrap does *not* perturb the cluster set; clusters are deterministic outputs of each method $\times$ hyperparameter cell.

Spurious rate — resample the clusters. Spurious rate is a proportion over the method’s *emitted clusters* (the denominator is `n_clusters`, which does not depend on the act set), so the act resample is the wrong sampling unit for it: dropping acts from the reference can only reclassify matched clusters as spurious, never the reverse, biasing the metric strictly upward and producing a CI that sits *entirely above* the point estimate. We therefore CI spurious rate with a separate cluster-level bootstrap: flag each cluster matched/unmatched against the full 49-act reference, then resample clusters with replacement. As an ordinary proportion bootstrap this centers the CI on the point. The interpretation shifts accordingly — the spurious CI answers “if the method had emitted a slightly different set of clusters” rather than “if R&R’s act list were a different draw.” Point estimates are identical to a single-pass computation under either reading; only the spurious CI columns reflect the cluster resample.

Worked example

Table 6 demonstrates the matching gates on one act and two contrasting clusters. The first cluster contains the canonical surface form ([Economic Recovery Tax Act of 1981](#)) and matches under both gates. The second is a hypothetical singleton cluster of just [ERTA](#): the keyword check fails (the subcomponent term is the full canonical name; “erta” does not contain “economic recovery tax act of 1981”), and the JW distance from “erta” to the canonical name is well above 0.30. An ERTA-only cluster therefore receives no credit — the method must merge ERTA with the canonical form for either gate to pass, which is exactly the aggregation behaviour C0 is meant to demonstrate.

Table 6: **Worked example.** Matching two hypothetical clusters against [Economic Recovery Tax Act of 1981](#) under the keyword and JW gates.

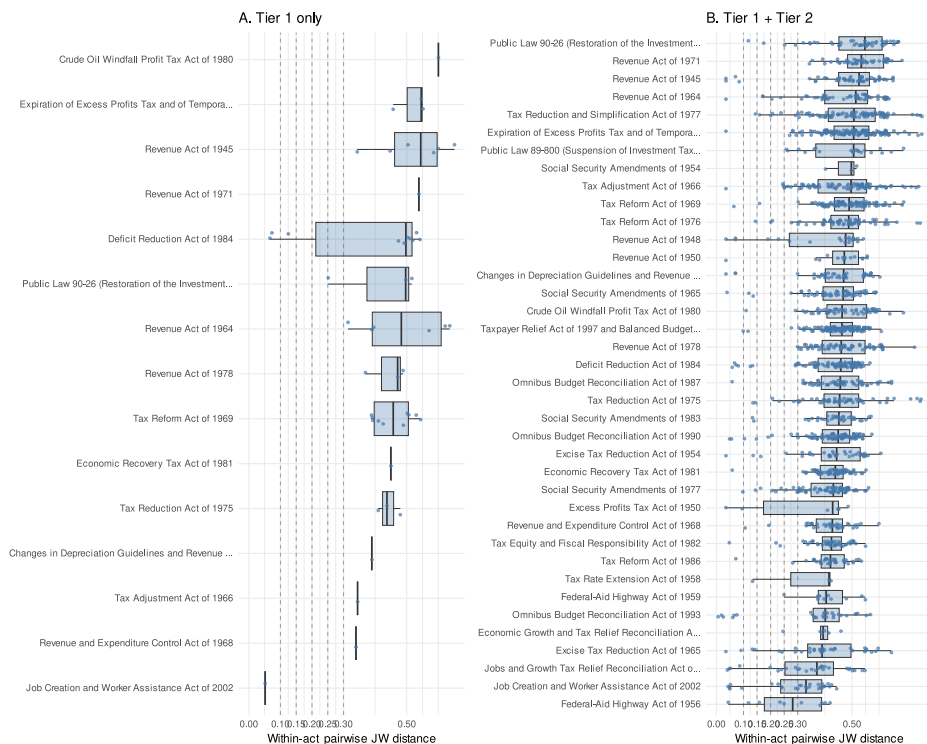
Cluster	mem-ber	Subcompo- nent terms	Keyword check	JW distance	JW $\leq$ 0.30
Economic Re- covery Tax Act of 1981	Re-	Economic Re- covery Tax Act of 1981	PASS	0.00	PASS
ERTA		Economic Re- covery Tax Act of 1981	FAIL	0.48	FAIL

Method 1 — JW single-linkage

Before reading the threshold sweep, Figure 5 shows what JW is being asked to recover. Each row is a gold act with ≥ 2 surface forms in the eval pool; the box and jittered points show the pairwise JW distances among the surface forms tagged to that act. Vertical dashes mark the swept thresholds (0.10–0.30). An act whose distribution sits entirely left of a threshold line is fully recoverable at that threshold; an act whose distances spread past 0.30 cannot be merged without

dragging in surface forms from neighbouring acts (i.e., over-merge). Ambiguous names are kept under every gold act they're tagged to, matching the descriptive intent of the figure rather than the evaluator's majority-vote.

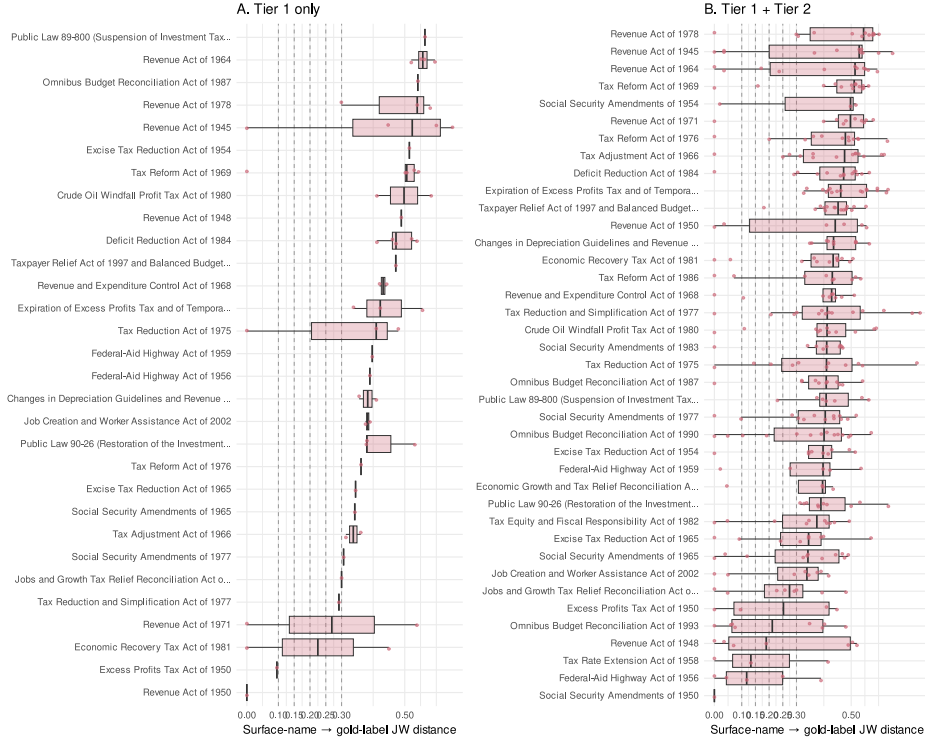
Figure 5: **Within-act JW distances**. Vertical dashes mark the swept thresholds. Acts with a single surface form contribute no pairs and are omitted.



What this means for Method 1. The median within-act JW distance is ~ 0.47 on both tiers, while the loosest threshold swept is 0.30. The typical pair of surface forms that *ought* to be merged is therefore geometrically out of reach for JW at any threshold the table considers; only a small minority of pairs (near-duplicates differing in spacing, punctuation, or abbreviation expansion) sit below 0.30. The implication is unavoidable: **JW single-linkage cannot canonicalize this pool on its own**. The threshold sweep in Table 7 below is best read as quantifying the asymmetric coverage/spurious cost given that fundamental ceiling, not as a search for a workable operating point. A method that can bridge [ERTA](#) $\leftrightarrow$ [Economic Recovery Tax Act of 1981](#) — semantic, not lexical — is required, which is what Phase B is for.

Figure 6 checks the same geometry from a different angle: instead of pairwise distances among surface forms, it measures each surface form's JW distance to its gold-act label. If C0 had the gold canonical list as an oracle and tried to map surface forms to it by JW, the swept thresholds tell us how many surface forms it would catch. The answer mirrors Figure 5: median ~ 0.41 on both tiers, well above 0.30. A handful of exact matches sit at 0 (surface form is the gold label string verbatim), but most surface forms are too lexically distant from their canonical to be matched without semantic reasoning.

Figure 6: **Surface-name** $\rightarrow$ **gold-label JW distance**. Same panels, threshold lines, and visual style as Figure 5; each point is one (measure_name, gold_act_name) pair.



The within-act figures pin down what JW can mechanically do; the ladders below quantify the consequences on the RR-aligned eval the \$Framing committed to. Each row is one variant (threshold $\times$ year-blocking); each cell carries a 95% bootstrap CI (act-indexed metrics resample the 49 RR acts, spurious rate resamples the clusters — see §“Bootstrap CI”). Column definitions are the headline four metrics defined in §“Evaluation against RR’s act list”:

- **Coverage** — share of 49 RR acts hit by ≥ 1 cluster under the joint name + year ± 2 gate.
- **Spurious** — share of clusters that match no RR act under the joint gate. The precision-flavoured column the asymmetric-cost framing weights most.
- **Fragmentation** — mean number of clusters claiming each matched RR act (≥ 1).
- **Year alignment** — diagnostic; share of name-matched pairs that also pass year ± 2 .

Two tables — one per name gate — so the keyword-vs-JW gap is visible side by side. Coverage will typically be higher under the JW gate (it accepts more surface forms); the keyword-vs-JW gap diagnoses how much of the JW coverage is recovery-by-near-canonical-match rather than recovery-by-aggregation.

Table 7: **JW single-linkage, keyword gate**. Cells: point estimate [95% bootstrap CI]. Rows: 5 thresholds $\times$ {Unblocked, Year ± 2 }. The keyword gate is name-rewriting-immune: a method gets credit only if a cluster member contains an RR-act subcomponent (from `generate_subcomponents()`).

JW thresh- old	Cover- age	Spuri- ous	Frag- menta- tion	Year align- ment	n clus- ters	ClusRR acts matched	
Unblocked							
0.10	0.673 [0.531, 0.796]	0.957 [0.947, 0.966]	2.212 [1.864, 2.550]	0.709 [0.635, 0.771]	1638	71	33
0.15	0.633 [0.490, 0.755]	0.963 [0.953, 0.971]	1.871 [1.550, 2.182]	0.659 [0.574, 0.736]	1502	56	31
0.20	0.612 [0.469, 0.735]	0.970 [0.961, 0.979]	1.500 [1.300, 1.667]	0.643 [0.561, 0.722]	1316	39	30
0.25	0.408 [0.265, 0.551]	0.977 [0.967, 0.986]	1.350 [1.154, 1.538]	0.435 [0.325, 0.526]	814	19	20
0.30	0.102 [0.020, 0.184]	0.994 [0.983, 1.000]	1.000 [1.000, 1.000]	0.114 [0.036, 0.179]	180	1	5
Year ± 2							
0.10	0.837 [0.714, 0.939]	0.948 [0.937, 0.958]	2.268 [1.958, 2.583]	0.886 [0.845, 0.929]	1707	89	41
0.15	0.837 [0.714, 0.939]	0.954 [0.944, 0.964]	1.878 [1.609, 2.125]	0.856 [0.810, 0.906]	1602	74	41
0.20	0.776 [0.653, 0.878]	0.962 [0.952, 0.971]	1.605 [1.400, 1.800]	0.803 [0.739, 0.868]	1465	56	38
0.25	0.755 [0.633, 0.857]	0.962 [0.950, 0.974]	1.432 [1.250, 1.591]	0.768 [0.692, 0.842]	1170	44	37
0.30	0.306 [0.184, 0.429]	0.978 [0.965, 0.989]	1.267 [1.100, 1.444]	0.345 [0.250, 0.432]	544	12	15

Table 8: **JW single-linkage, JW-min $\leq$ 0.30 gate.** Same variants and column definitions as Table 7. Continuous distance; accepts near-canonical variants the keyword gate misses. The Coverage gap from Table 7 is the relaxation bonus.

JW thresh- old	Cover- age	Spuri- ous	Frag- menta- tion	Year align- ment	n clus- ters	ClusRR acts ters matched	ClusRR acts matched
Unblocked							
0.10	0.939 [0.877, 1.000]	0.873 [0.857, 0.889]	6.587 [5.656, 7.435]	0.181 [0.161, 0.204]	1638	208	46
0.15	0.918 [0.837, 0.980]	0.885 [0.870, 0.901]	5.600 [4.806, 6.385]	0.173 [0.154, 0.194]	1502	172	45
0.20	0.898 [0.796, 0.980]	0.894 [0.878, 0.910]	4.773 [4.034, 5.429]	0.169 [0.148, 0.190]	1316	140	44
0.25	0.694 [0.571, 0.816]	0.924 [0.905, 0.942]	2.559 [2.095, 3.000]	0.176 [0.147, 0.207]	814	62	34
0.30	0.163 [0.061, 0.265]	0.983 [0.961, 1.000]	1.000 [1.000, 1.000]	0.143 [0.062, 0.205]	180	3	8
Year ± 2							
0.10	0.939 [0.877, 1.000]	0.864 [0.848, 0.880]	7.543 [6.586, 8.429]	0.179 [0.161, 0.201]	1707	233	46
0.15	0.939 [0.877, 1.000]	0.873 [0.857, 0.888]	6.696 [5.871, 7.500]	0.173 [0.155, 0.193]	1602	204	46
0.20	0.939 [0.877, 1.000]	0.878 [0.861, 0.894]	6.022 [5.167, 6.828]	0.170 [0.152, 0.189]	1465	179	46
0.25	0.939 [0.877, 1.000]	0.887 [0.870, 0.905]	4.435 [3.833, 5.067]	0.158 [0.143, 0.176]	1170	132	46
0.30	0.633 [0.510, 0.755]	0.949 [0.928, 0.967]	1.516 [1.347, 1.684]	0.129 [0.104, 0.154]	544	28	31

Figure 7 plots the two ladder tables as threshold sweeps so the trade-offs read at a glance. Each panel column is a name gate (keyword vs JW-min $\leq$ 0.30), each row is one headline metric (with its greater/lower-is-better direction in the

strip label), and the two lines per panel are the unblocked vs year- ± 2 variants. Bands are the 95% bootstrap CIs (see §“Bootstrap CI”; spurious rate is cluster-resampled, the other three act-resampled). Read the coverage row against the spurious-rate row at each threshold: the asymmetric-cost framing wants the threshold that keeps spurious low without collapsing coverage.

Figure 7: **JW headline metrics vs threshold.** Columns: name gate (keyword vs JW-min ≤ 0.30). Rows: the four headline metrics, with $\uparrow$ (greater is better) / $\downarrow$ (lower is better) in the strip label. Colour: unblocked vs year- ± 2 blocking. Bands are 95% bootstrap CIs (spurious rate cluster-resampled, the other three act-resampled; see §Bootstrap CI). Y-axes are free per metric row.

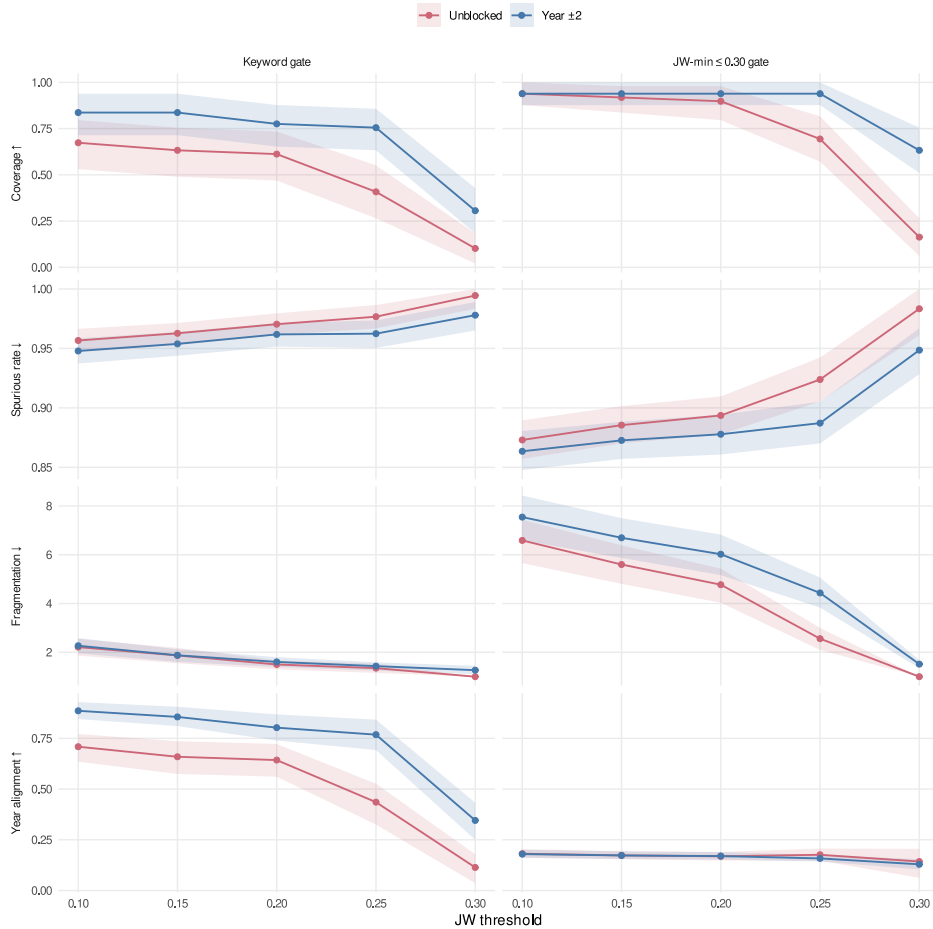
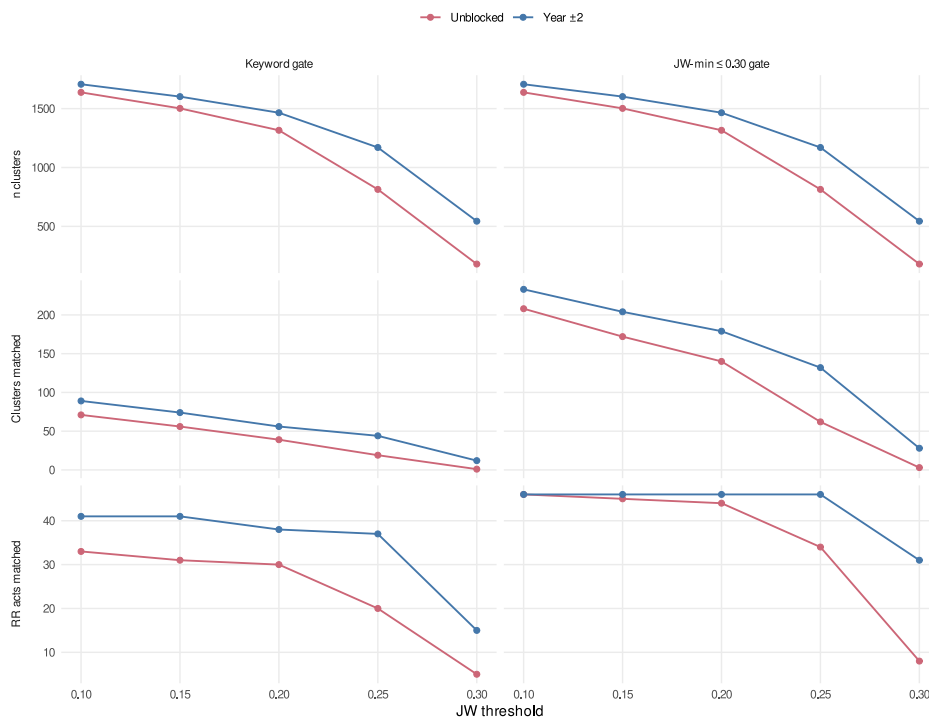


Figure 8 shows the raw counts behind those rates on the same axes: the cluster budget the method emits (n clusters), how many of those carry an RR match (Clusters matched), and how many of the 49 RR acts get covered (RR acts matched). These are the numerator and denominator pieces the headline rates are built from, so the figure makes the mechanism legible: the spurious rate is high because n clusters runs into the thousands while Clusters matched stays in the dozens. n clusters is identical across the two gate columns by construction (it depends on the clustering, not the matching gate); only the lower two rows respond to the gate.

Figure 8: **JW match counts vs threshold**. Same layout as Figure 7 but plotting raw counts rather than rates. Columns: name gate. Rows: total clusters emitted, clusters carrying an RR match, and RR acts covered (out of 49). Colour: unblocked vs year- ± 2 blocking. Y-axes are free per row; `n_clusters` is gate-invariant so its two columns are identical. Counts are exact (no bootstrap CI).



Reading the match counts

Three mechanics make the count rows legible, and each corrects an intuition the rates alone invite.

First, **year- ± 2 blocking lifts `n_clusters` at every threshold because it forbids merges, not because it drops names**. Blocking is a clustering-side constraint: only name pairs within ± 2 years are eligible to merge (`run_jw_clusters` masks the rest). No name leaves the pool, so the count cannot fall. Pairs that JW distance alone would have fused but that sit more than two years apart stay separate, so consolidation drops and the cluster count rises. The natural guess that “filtering should shrink the output” has the sign backwards: same elements, fewer permitted merges, more clusters. Note also that blocking’s lift on the lower two rows (`Clusters matched`, `RR acts matched`) is a *different* effect from its lift on `n_clusters`: the extra clusters come from forbidden merges, but the extra matches come from clusters staying year-coherent enough to clear the year gate.

Second, **the coverage collapse at high thresholds is a year-gate failure, not names going missing**. A cluster name-matches on *any* of its member surface forms (minimum JW distance, any keyword hit across members), so a blob that swallows two acts’ names still name-matches both. What it cannot preserve is a single representative year: the cluster year is the median of its members’ years, and single-linkage chaining at a loose threshold smears that median across a

wide span. Acts whose true year drifts more than two years from the blob's median then fail the year gate and drop out of `RR acts matched`. The decisive evidence is in the figure itself: at $t = 0.30$ the unblocked variant covers 8 acts while year- ± 2 blocking covers 31. Blocking touches only the year-eligibility of merges, so a 4 $\times$ swing can only be the year gate at work. (Even the blocked variant slips from 46 to 31 at $t = 0.30$, because ± 2 pairwise eligibility still lets a single-linkage chain span more than two years end to end.)

Third, **Clusters matched falls with the threshold for two compounding reasons, only one of which is overmerging.** By construction `Clusters matched` = `n clusters` $\times$ $(1 - \text{spurious rate})$, and both factors shrink as the threshold loosens: the cluster budget collapses ($\approx 1,640$ down to ≈ 180 in the unblocked JW gate) and the match rate falls (about 13% down to under 2%). The overmerge-and-year-smear story explains the rate term; the shrinking cluster budget, the larger arithmetic factor, explains the count term. Reading the whole fall as overmerging would miss that most of it is simply far fewer clusters left to match.

Interpreting the threshold sweep

Read across the four rows, the conservative end (JW threshold 0.10) is weakly best on three of the four metrics and worst on the fourth. Coverage, spurious rate, and year alignment all favour the low threshold; fragmentation favours the high one. That single crossing is the whole trade-off, and it is consistent with fragmentation being a consolidation (over-segmentation) metric that rewards aggressive merging while the other three reward keeping clusters fine-grained. Note that the recall-flavoured metric here is *coverage* (did we recover all 49 acts), not fragmentation.

But “0.10 dominates” overstates what the sweep shows, and three qualifications matter:

- **The dominance is mostly inside the noise.** Under year- ± 2 blocking, coverage is flat at 0.939 from $t = 0.10$ through 0.25 (JW gate), so 0.10 is tied, not ahead; unblocked, the coverage CIs are wide (act-resampled, $n = 49$) and overlap heavily across 0.10–0.20. Year alignment is the same story, and its real driver is the *gate*, not the threshold: the JW gate sits near 0.18 at every threshold while the keyword gate runs 0.7–0.9 at low thresholds (crashing toward 0.1–0.35 as the threshold loosens). The only metric on which 0.10 is cleanly best is the spurious rate, whose tight cluster-resampled CIs barely overlap.
- **The spurious “win” is least-bad, not good, and partly structural.** The spurious rate is 0.86–0.99 everywhere; even the best cell leaves $\sim 86\%$ of clusters matching no RR act. That floor is largely a target-size mismatch: the pool holds 1,976 distinct measure names but R&R's exogenous list is only 49 acts, so most clusters are legitimately non-RR measures. Optimising a metric that is structurally pinned near 1 reads more like choosing the least-bad column than recovering precision.

- **The fragmentation improvement is confounded with collapsing coverage.** Fragmentation is averaged only over *matched* acts. At $t = 0.30$ it reaches 1.00, but only 8 of 49 acts match there, so the denominator has shrunk to the few easy cases. The apparent consolidation gain is partly coverage collapsing onto its cleanest survivors, not merging getting better.

The interpretation to confront is that **$t = 0.10$ is close to the identity transform**. It collapses only about 17% of the pool’s 1,976 distinct names into the 1,638 clusters it emits, so “conservative threshold wins” comes dangerously close to “do almost no aggregation and you cover the most acts.” Read skeptically, the sweep is not evidence that JW recovers truth best when conservative; it is evidence that the JW step is barely earning its keep, with the raw measure pool already covering ~46/49 acts and every merge only costing coverage.

Whether the conservative choice is genuinely right therefore depends on the asymmetric cost we adopted: over-merging (collapsing two distinct RR acts into one cluster) is the costly, unrecoverable error, while under-merging (fragmentation) is cheap and recoverable downstream. Under that cost model, accepting high fragmentation at $t = 0.10$ to protect coverage is defensible, but only because fragmentation has been explicitly priced as cheap. The conclusion is conditional on that pricing, not unconditional. Year- ± 2 blocking reinforces it: by flattening the coverage curve it makes the threshold choice nearly irrelevant for coverage, leaving 0.10’s edge resting on the spurious column alone.

Reading the JW results

One reading the threshold and count subsections above do not cover is the **keyword-vs-JW gate gap** as a diagnostic. The $\text{JW-min} \leq 0.30$ gate accepts more surface forms than the keyword gate, so it always covers more RR acts: by roughly 27–29 percentage points unblocked ($t = 0.10$ – 0.25 , falling to 6 pp at $t = 0.30$) and 10–18 points under year- ± 2 blocking (in raw acts, 3–16 of 49). That gap measures how much of JW’s coverage is *recovery-by-near-canonical-match* — typographic variants and short abbreviations within JW distance 0.30 — rather than *recovery-by-aggregation*. Because JW single-linkage never mints a canonical name (it just keeps whatever surface forms it sees), the bonus here is almost entirely near-duplicate recovery, not renaming.

This is the comparison the benchmark exists to enable. A stateful method that emits one canonical name per cluster (Method 5) should open this gate gap much wider, because its coverage will come from genuine renaming rather than lexical luck. The JW ladder is the floor that gain is measured against.

Bottom line for Method 1

JW single-linkage cannot canonicalise this measure pool on its own. The geometric ceiling from Figure 5 is binding: most of the surface-form pairs that ought to merge sit beyond JW distance 0.30, geometrically out of reach for JW at any threshold considered. The RR-aligned ladder confirms it: at every variant the spurious rate stays above 86%, fragmentation ranges from 1.0 to 7.5 (1.5–3 under the keyword gate, but 4.8–7.5 under the JW gate at conservative thresholds), and

even the best variant misses ~8 of 49 RR acts under the keyword gate. The gap between *ERTA* and *Economic Recovery Tax Act of 1981* is semantic, not lexical, and JW has no machinery to bridge it.

What JW remains useful for is the role its mechanics actually support: a precision-friendly **blocker** — a cheap pre-filter that catches near-duplicate surface forms (spacing, punctuation, abbreviation expansion) at near-zero false-positive cost. Method 4’s hybrid pipeline below uses exactly this property: JW (or a cheap analog) prunes the pair space, and a semantic verifier handles the merges that surface similarity cannot. The load-bearing canonicalisation work, though, has to come from a method that understands meaning — Methods 2–5 are the candidate replacements.

Embedding-geometry probe (F16 vs FP32)

Method 1 closed on the observation that the gap to canonicalise this pool is semantic, not lexical. The next three sections test whether multilingual embeddings can bridge it. Before reading the eval, one preliminary: the Methods 2/3 embeddings come from a community F16 port of `intfloat/multilingual-e5-large-instruct` (`jeffh/intfloat-multilingual-e5-large-instruct:f16`) running locally under Ollama. F16 compresses the reference FP32 weights and could in principle distort the embedding geometry that HDBSCAN relies on. Two cheap probes on the gold eval pool check that the port preserves the semantic structure HDBSCAN needs.

The probes run on a 2×2 grid: **precision** (F16 local vs FP32 reference via Deep-Infra) × **eval slice** (full unambiguous pool vs Tier 1 only). The FP32 reference disentangles two distinct failure modes the F16-only probe couldn’t separate: “F16 quantization broke the geometry” (FP32 passes where F16 fails) vs “the model itself can’t do this task” (both precisions fail). The Tier 1 slice in turn strips Tier 2’s permissive IKA tagging, which §“Eval-set noise probe” below shows is genuinely noisy. Reading the four cells together discriminates quantization noise, model-capacity ceiling, and label noise.

Table 9: **Embedding-geometry probe.** A 2×2 (precision × eval slice) grid. Top-1 NN same-act rate is the share of eval names whose nearest-neighbour by cosine shares a gold act (restricted to acts with ≥2 members so the metric is well-defined). Separation = median within-act cosine – median between-act cosine; positive means the embedding preserves the act-level structure. F16 rows come from the local Ollama port ([jeffh/intfloat-multilingual-e5-large-instruct:f16](#)); FP32 rows from DeepInfra-hosted [intfloat/multilingual-e5-large-instruct](#). Same instruction prefix on both.

Preci- sion	Eval slice	Top-1 NN same- act rate	Me- dian within- act co- sine	Me- dian be- tween- act co- sine	Sepa- ration	Eval names	Eval acts
F16 (lo- cal Ol- lama)	Full pool (Tier 1 + 2)	0.261	0.903	0.898	0.005	206	38
F16 (lo- cal Ol- lama)	Tier 1 only	0.308	0.919	0.902	0.017	39	22
FP32 (Deep- Infra)	Full pool (Tier 1 + 2)	0.251	0.903	0.898	0.005	206	38
FP32 (Deep- Infra)	Tier 1 only	0.269	0.919	0.902	0.017	39	22

F16, Tier 1: separation 0.017 (floor 0.05), top-1 NN rate 0.308 (floor 0.40) — **FAIL**
 FP32, Tier 1: separation 0.017 (floor 0.05), top-1 NN rate 0.269 (floor 0.40) — **FAIL**

Reading Tier 1 vs Tier 1 keeps the gold-partition noise floor constant on both rows so the F16↔FP32 contrast is apples-to-apples.

Verdict — embedding-model ceiling, not quantization. Both F16 and FP32 fail the same floors, so the issue is not F16 specifically — [multilingual-e5-large-instruct](#) itself does not produce the relative cosine spread the floors require on this pool. Two interpretations to weigh: (i) E5-instruct embeddings live in a tight band of the unit sphere by design, and the 0.05/0.40 floors may be inappropriate for this family; (ii) the model is genuinely under-resolving fiscal-act semantics and a stronger or differently-tuned embedding family is needed. Either way, the FP32 migration is *not* the fix. Read Methods 2/3 below as the empirical end-to-end test — if HDBSCAN’s Tier 1 numbers are competitive with JW, the floors were the over-strict reading.

E5-family embeddings always live in a narrow band of the unit sphere, so absolute cosines aren't crisp — the probe targets the *relative* gap between within-act and between-act pairs, which is the property HDBSCAN cares about. Methods 2 and 3 below continue to read `c0_us_embeddings` (the F16 matrix); the FP32 numbers above are diagnostic only.

Eval-set noise probe: surface-year vs doc-year mismatch

An alternative explanation for a soft F16 verdict — and a noise source the within-act geometry from §Method 1 already hinted at — is that the gold pairs themselves are temporally noisy. If C1 sometimes catches contemporary discussions of acts passed decades earlier (e.g. a 2010 ERP paragraph referring back to the Revenue Act of 1964), the resulting `(measure_name, gold_act_name)` pair will look lexically distant — not because the model misclassified, but because the surface form drifts as the act becomes historical context rather than current policy. Figure 9 probes this directly: for each eval tuple whose surface form contains a 4-digit year, plot the gap between the document's publication year and the year referenced in the surface form against the JW distance to the gold label.

Figure 9: **Temporal mismatch in eval pairs.** Each point is one `(measure_name, gold_act_name, chunk_id)` tuple whose surface form contains a 4-digit year. $x = \text{doc publication year} - \text{year referenced in the surface form}$ (large positive $\Rightarrow$ retrospective reference); $y = \text{JW distance from surface form to gold label}$. Tuples whose surface form has no extractable year are omitted.

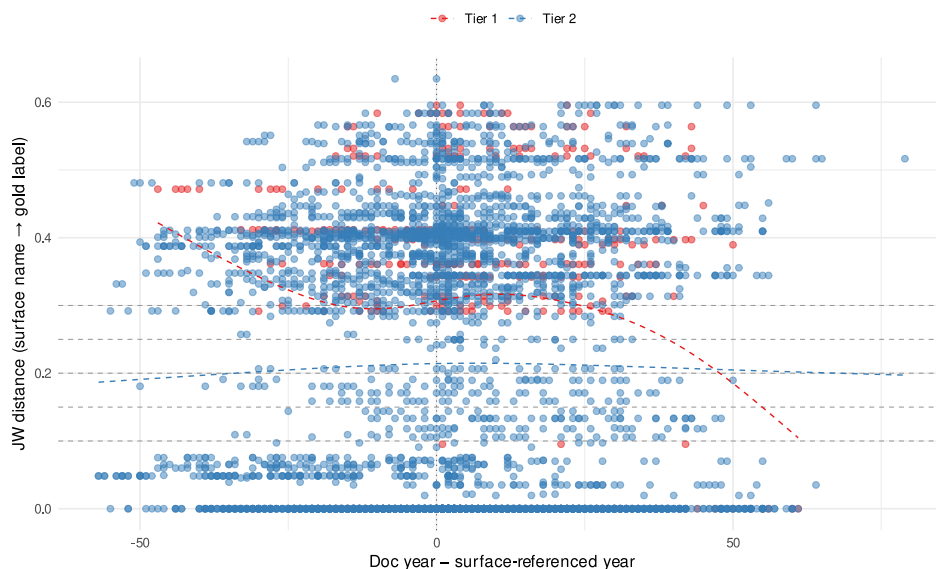
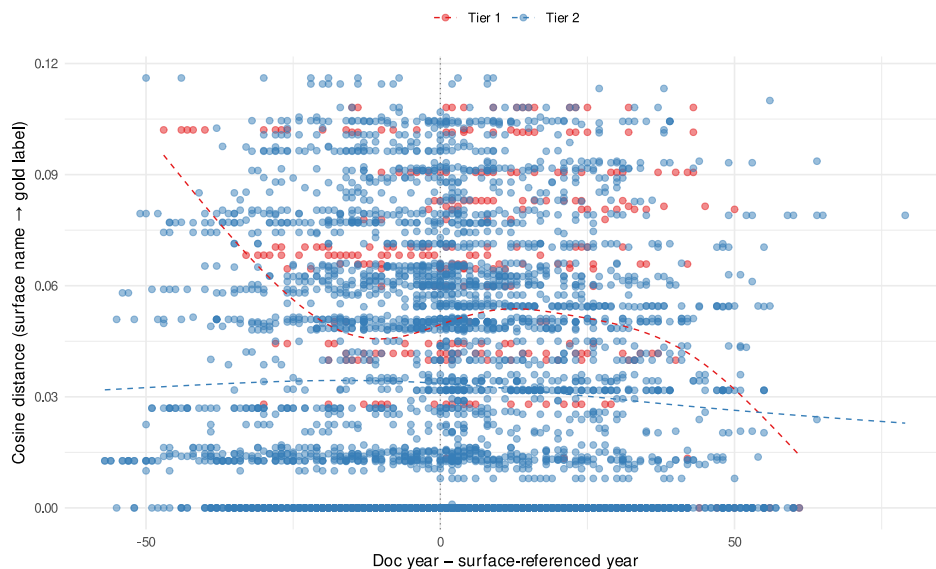


Figure 10 repeats the panel on the semantic axis: y is cosine distance between the surface-name and gold-label embeddings (E5-multilingual F16, L2-normalised — so cosine distance = $1 - \text{dot product}$, on the same scale as the F16 probe). The interpretation flips relative to JW: if retrospective references drift *only* lexically (paraphrase) but stay semantically anchored, the cloud should stay flat as x

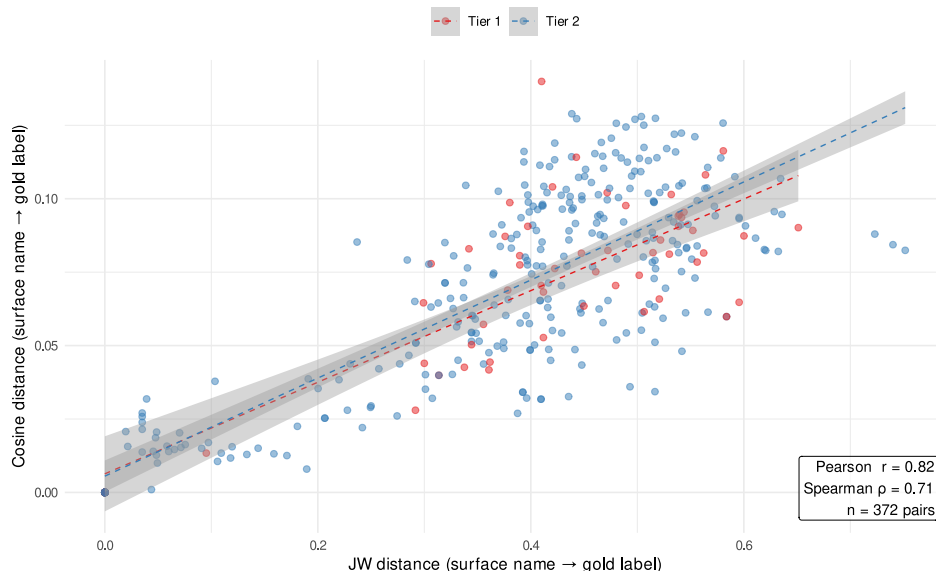
grows; if they drift semantically as well (different terminology or focus when describing the act decades later), the cloud should rise. Reading the two figures together separates the two failure modes.

Figure 10: **Eval-pair temporal mismatch (cosine)**. Same panel as Figure 9 but with cosine distance (E5-multilingual F16 embeddings, L2-normalised) on y. Pairs are dropped when either the surface form or the gold label is absent from its respective embedding matrix.



Finally, Figure 11 puts the two distance measures head-to-head on the full eval pool (no year filter): one point per distinct (`measure_name`, `gold_act_name`) pair, x is the JW distance, y is the cosine distance. The correlation answers whether the embedding is buying us anything JW couldn't already give us — a strong positive correlation would mean E5 is essentially re-deriving lexical similarity, while a weak correlation means the two distance measures disagree on which pairs are similar, and there is genuine semantic signal in the embedding above and beyond surface overlap. Both Pearson (linear) and Spearman (rank) correlations are reported because the JW–cosine relationship is plausibly nonlinear.

Figure 11: **JW vs cosine distance**. For every distinct (measure_name, gold_act_name) pair in the eval pool. Colour marks the cleanest tier in which the pair appears (Tier 1 if at least one chunk pairs the surface form with the gold act via verbatim match). Pairs whose surface form or gold label is missing from its embedding matrix are dropped. Annotated correlations include the full pool; the dashed line is an unweighted linear fit.



Method 2 — Multilingual embedding + HDBSCAN

Switching from string distance to embedding distance is what the within-act geometry from §Method 1 demanded. Method 2 embeds each measure name with `intfloat/multilingual-e5-large-instruct`, L2-normalises the vectors so cosine ranking equals euclidean ranking, then runs HDBSCAN over the resulting distance matrix. Three `min_cluster_size` values are swept — 2 (most permissive), 3, and 5 (close to the median gold-act size). Singleton “noise” points HDBSCAN refuses to merge are kept as their-own-clusters; under the RR-aligned eval they contribute to the spurious rate denominator (singleton clusters with no RR-act match raise spurious; singletons that *do* contain a canonical name still match).

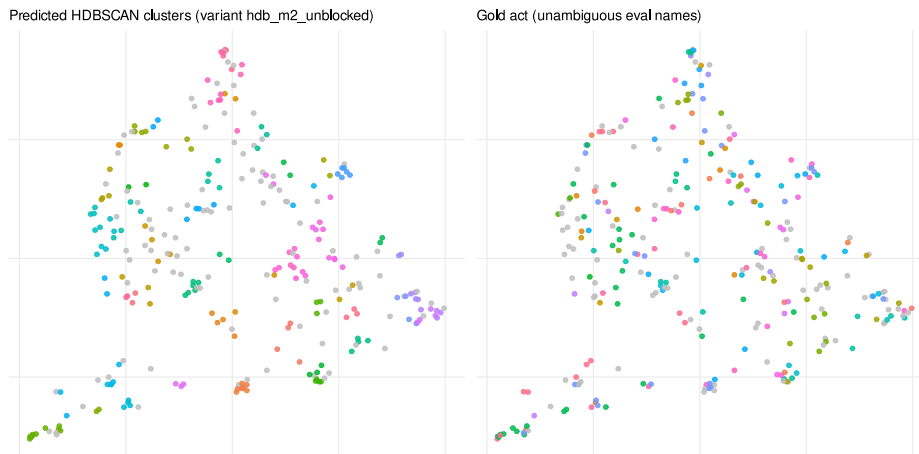
Table 10: **HDBSCAN unblocked, keyword gate.** Cells: point estimate [95% bootstrap CI]. Rows: 3 `min_cluster_size` settings. Compare to Table 7 — if embeddings buy us what JW couldn’t, coverage should hold and spurious should drop.

<code>min_cluster_size</code>	Coverage	Spurious	Fragmentation	Year alignment	n clusters	ClusRR acts matched	ters matched
2	0.592 [0.449, 0.714]	0.822 [0.763, 0.876]	1.103 [1.000, 1.176]	0.762 [0.654, 0.875]	169	30	29
3	0.408 [0.265, 0.531]	0.845 [0.790, 0.895]	1.450 [1.200, 1.700]	0.547 [0.429, 0.643]	181	28	20
5	0.449 [0.306, 0.592]	0.825 [0.777, 0.879]	1.682 [1.385, 1.923]	0.627 [0.500, 0.725]	206	36	22

Table 11: **HDBSCAN unblocked, JW-min ≤ 0.30 gate.** Same variants as Table 10. The keyword-vs-JW gap diagnoses recovery-by-aggregation vs. recovery-by-canonical-form.

<code>min_cluster_size</code>	Coverage	Spurious	Fragmentation	Year alignment	n clusters	ClusRR acts matched	ters matched
2	0.918 [0.837, 0.980]	0.686 [0.621, 0.757]	2.022 [1.767, 2.267]	0.192 [0.166, 0.216]	169	53	45
3	0.776 [0.653, 0.898]	0.773 [0.713, 0.829]	1.816 [1.591, 2.036]	0.214 [0.180, 0.248]	181	41	38
5	0.816 [0.714, 0.918]	0.738 [0.680, 0.796]	2.075 [1.783, 2.381]	0.226 [0.191, 0.262]	206	54	40

Figure 12: **UMAP projection of E5 embeddings**. Left: coloured by predicted HDBSCAN cluster (best unblocked variant by Tier 1 F1); singleton/noise names in grey. Right: coloured by gold act for names in the unambiguous eval pool; names outside the eval pool in grey. Same 2D coordinates in both panels.



Reading the HDBSCAN results

The headline comparison is Table 10 against Table 7: if embeddings buy us what JW couldn’t, HDBSCAN should hold or improve coverage while collapsing the spurious-rate gap. The UMAP scatter Figure 12 is diagnostic — predicted clusters that overlap a single gold colour on the right panel are correct merges; predicted clusters spanning multiple gold colours are over-merges (note: the gold panel uses `c0_eval_gold_pairs` so the colour palette inherits the gold-pool noise documented in §“Why we pivoted away from pairwise P/R” — read it as a relative-structure visualisation, not as ground truth on which clusters are correct). The §Synthesis at the bottom of the notebook applies the precision-friendly shortlist rule (lowest spurious, then highest coverage) to pick a candidate variant across methods.

Method 3 — Year-blocked variants

Method 3 reruns the same HDBSCAN sweep with the year-window mask from §Method 1: pairs whose chunk years differ by more than 2 are forced to maximum distance and so cannot merge. The prior is that the same act tends to surface in a narrow temporal window, so the mask should reduce cross-decade over-merges without sacrificing coverage.

Partial redundancy with the match gate. The RR-aligned eval already applies a year ± 2 window between cluster representative year and `year_signed`, so a cluster that *would* have over-merged across decades will fail the year gate anyway and not match the wrong act. Year-blocking at clustering time and year ± 2 at match time are not the same operation, though: clustering-time blocking prevents the merge from happening (the cluster doesn’t form), while match-time blocking lets the merge happen but disqualifies the resulting cluster from claiming an act of the wrong year. So year-blocking still helps by reducing the *count* of cross-decade clusters that would otherwise raise the spurious rate

denominator. The expected pattern: spurious rate drops modestly under year-blocking; coverage and fragmentation hold.

Table 12: **HDBSCAN year-blocked, keyword gate.** Year-blocked (± 2 at clustering). Same column definitions as Table 10.

min_cluster_size	coverage	Spurious	Fragmentation	Year alignment	n clusters	Clusters matched	RR acts matched
2	0.755 [0.633, 0.857]	0.783 [0.726, 0.846]	1.108 [1.040, 1.182]	0.976 [0.955, 1.000]	175	38	37
3	0.673 [0.531, 0.796]	0.807 [0.746, 0.862]	1.303 [1.182, 1.429]	0.878 [0.815, 0.941]	181	35	33
5	0.224 [0.122, 0.347]	0.898 [0.816, 0.959]	1.000 [1.000, 1.000]	0.289 [0.178, 0.409]	49	5	11

Table 13: **HDBSCAN year-blocked, JW-min ≤ 0.30 gate.** Year-blocked (± 2 at clustering).

min_cluster_size	coverage	Spurious	Fragmentation	Year alignment	n clusters	Clusters matched	RR acts matched
2	0.918 [0.837, 0.980]	0.680 [0.606, 0.749]	2.222 [1.903, 2.519]	0.188 [0.164, 0.212]	175	56	45
3	0.898 [0.796, 0.980]	0.724 [0.657, 0.785]	1.955 [1.692, 2.214]	0.173 [0.152, 0.196]	181	50	44
5	0.347 [0.224, 0.490]	0.857 [0.755, 0.939]	1.000 [1.000, 1.000]	0.114 [0.083, 0.148]	49	7	17

Compare each row to its unblocked counterpart in Table 10 / Table 11. If year-blocking lowers spurious rate at the same min_cluster_size with comparable coverage, the temporal prior remains structural across the eval pivot and is worth carrying into Methods 4–5 when they land.

Hyperparameter sweep (Methods 2 + 3)

The four no-UMAP tables above are six variants — min_cluster_size $\in \{2, 3, 5\}$ crossed with {unblocked, year ± 2 }. Figure 13 and Figure 14 plot that sweep in the same idiom as Figure 7 / Figure 8 so Methods 2 + 3 read on the same axes as Method 1: each panel column is a name gate (keyword vs JW-min ≤ 0.30), each row is an outcome, the x-axis is the only swept hyperparameter

(`min_cluster_size`), and colour is year-blocking. The HDBSCAN counterpart of the UMAP grid's four-knob scan (Figure 16) is degenerate here — one knob, three points.

Figure 13: **HDBSCAN headline metrics vs `min_cluster_size`**. Methods 2 + 3 (no UMAP). Columns: name gate (keyword vs JW-min ≤ 0.30). Rows: the four headline metrics, with $\uparrow / \downarrow$ in the strip label. x: the swept `min_cluster_size`. Colour: unblocked vs year- ± 2 blocking. Bands are 95% bootstrap CIs (spurious rate cluster-resampled, the other three act-resampled; see §Bootstrap CI). Y-axes free per metric row.

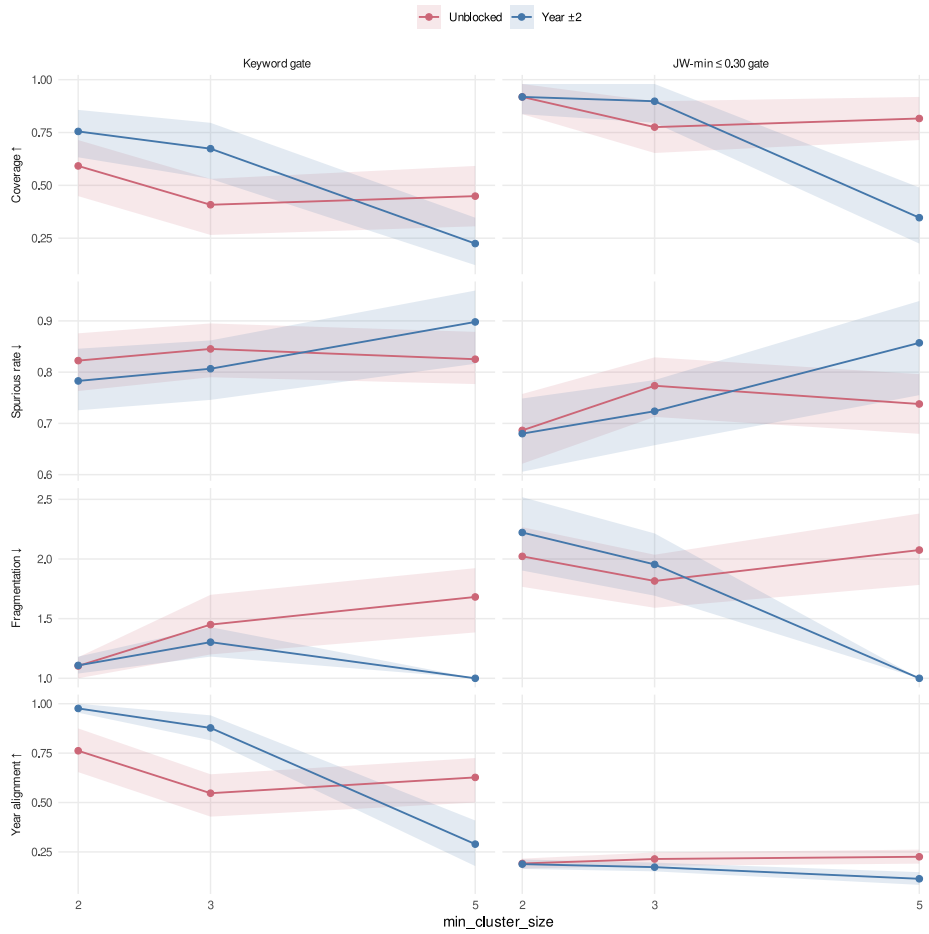
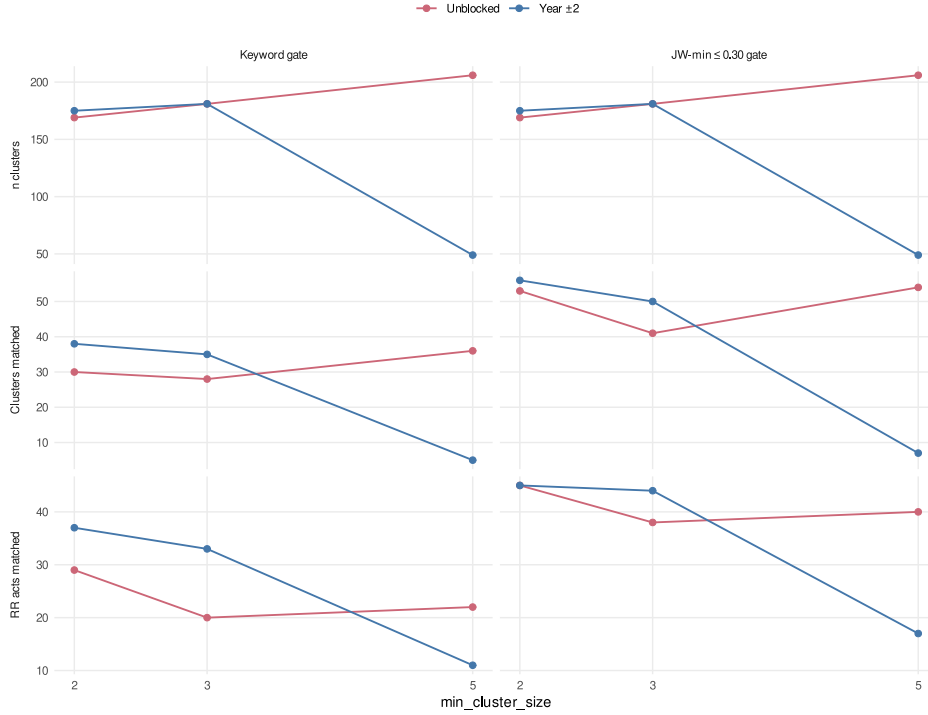


Figure 14: **HDBSCAN match counts vs min_cluster_size**. Same layout as Figure 13 but plotting raw counts. Columns: name gate. Rows: total clusters emitted, clusters carrying an RR match, RR acts covered (out of 49). `n_clusters` is gate-invariant so its two columns are identical; only the lower two rows respond to the gate. Counts are exact (no bootstrap CI).



Methods 2 + 3 with UMAP dimensionality reduction

The F16 and FP32 embedding-geometry probes above both failed the `separation` ≥ 0.05 and `top-1 NN same-act rate` ≥ 0.40 floors on the gold pool. The verdict text considered two interpretations — F16 quantization noise, and the embedding-model ceiling — but did not weigh a third: **high-dimensional distance concentration**. `intfloat/multilingual-e5-large-instruct` returns 1024-dimensional vectors, and the unblocked HDBSCAN above operates on the full-dimensional cosine distance matrix. Density-based clustering is known to degrade above $k \approx 50$ dim through hubness and farthest-to-nearest ratio compression; the standard fix in the BERTopic / Top2Vec neighbourhood is to interpose UMAP between the embedding and the clusterer, collapsing the ambient dim to $\sim 5-10$ and (optionally) packing within-cluster points tightly via `min_dist = 0`. This section ports that recipe into Methods 2 + 3 and reads the with-vs-without delta against the no-UMAP baseline above.

The sweep grid is the cartesian product of three UMAP hyperparameters with the existing HDBSCAN grid:

- `n_neighbors` $\in \{5, 15, 30\}$ — uwot default 15; the small end emphasises local structure, the large end emphasises global structure.

- `n_components` $\in \{2, 5, 10\}$ — uwot default 2; BERTopic’s standard is 5; 10 is the upper end of clustering practice.
- `min_dist` $\in \{0.0, 0.01, 0.1\}$ — uwot default 0.01; 0.0 is BERTopic’s tight-packing default; 0.1 matches the existing diagnostic UMAP plot at Figure 12.
- Inner sweep matches the no-UMAP variants: `min_cluster_size` $\in \{2, 3, 5\}$ and `year_window` $\in \{\text{unblocked}, \pm 2\}$.

That gives 27 UMAP cells $\times$ 6 HDBSCAN cells = 162 variants. Metric "cosine" is fixed (the input embeddings are L2-normalised); UMAP is reduced once per UMAP cell and reused across the inner HDBSCAN sub-cells. The seed is fixed at the target level (`run_hdbscan_umap_clusters_grid(seed = 20260526L)`) so the entire sweep is deterministic under `tar_make()`. Distance metric inside HDBSCAN switches from cosine on the raw embeddings to Euclidean on the UMAP output (UMAP does not preserve L2-normalisation, so cosine on the reduced space is not meaningful); the year-window mask uses a `max(d) + 1` sentinel instead of the cosine 1.0 cap.

Table 14: **Best UMAP cell per sub-cell, keyword gate.** Best UMAP cell per (`min_cluster_size` $\times$ year-blocking) sub-cell. Within each sub-cell we sort by `spurious_rate` ascending then coverage descending and report the winner. Cells: point estimate [95% bootstrap CI]. Compare each row to the corresponding no-UMAP row in Table 10 / Table 12 — if UMAP wins, the cells should sit at lower spurious with comparable-or-higher coverage.

	<code>min_cluster_size</code>	<code>n_neighbors</code>	<code>n_components</code>	<code>min_dist</code>	Cov- er- age	Spu- ri- ous	Frag- men- ta- tion	Year align- ment	n clus- ters
unblocked	2	5	2	0.010	0.673 [0.634, 0.714]	0.548 [0.514, 0.582]	1.083 [1.030, 1.127]	0.930 [0.758, 1.04]	104
unblocked	3	5	10	0.010	0.571 [0.429, 0.714]	0.497 [0.467, 0.527]	1.071 [1.001, 1.133]	0.714 [0.609, 0.815]	67
unblocked	5	5	2	0.010	0.429 [0.366, 0.571]	0.506 [0.504, 0.508]	1.051 [1.051, 1.051]	0.615 [0.409, 0.815]	45
year ± 2	2	15	5	0.000	0.714 [0.692, 0.837]	0.528 [0.520, 0.536]	1.091 [1.091, 1.091]	0.913 [0.852, 1.17]	117
year ± 2	3	5	10	0.000	0.673 [0.633, 0.796]	0.563 [0.561, 0.565]	1.083 [1.083, 1.083]	0.926 [0.760, 0.926]	92
year ± 2	5	5	5	0.000	0.612 [0.463, 0.735]	0.533 [0.533, 0.533]	1.080 [1.080, 1.080]	0.879 [0.714, 0.93]	93

Table 15: **Best UMAP cell per sub-cell, JW gate.** Same shortlist rule on the JW-min ≤ 0.30 gate. The keyword-vs-JW gap is the recovery-by-near-canonical bonus.

Block	cluster	size	neighbors	components	dist	Cov- er- age	Spu- rious	Frag- men- ta- tion	Year align- ment	n clus- ters
unblocked	2	5	5	0.100	0.918 [0.832, 0.980]	0.522 [0.441, 0.611]	1.141 [1.048, 1.237]	0.158, 113	0.208]	
unblocked	3	5	10	0.010	0.735 [0.643, 0.857]	0.123 [0.102, 0.552]	1.028 [1.027, 1.029]	0.143, 67	0.209]	
unblocked	5	5	2	0.010	0.612 [0.496, 0.735]	0.167 [0.170, 0.600]	1.170 [1.117, 1.220]	0.133, 45	0.209]	
year ± 2	2	15	2	0.010	0.898 [0.853, 0.980]	0.538 [0.443, 0.630]	1.132 [1.008, 1.256]	0.166, 119	0.214]	
year ± 2	3	5	10	0.000	0.857 [0.755, 0.939]	0.554 [0.457, 0.652]	1.078 [1.092, 1.063]	0.151, 92	0.211]	
year ± 2	5	5	5	0.000	0.816 [0.748, 0.918]	0.548 [0.440, 0.645]	1.110 [1.040, 1.180]	0.145, 93	0.203]	

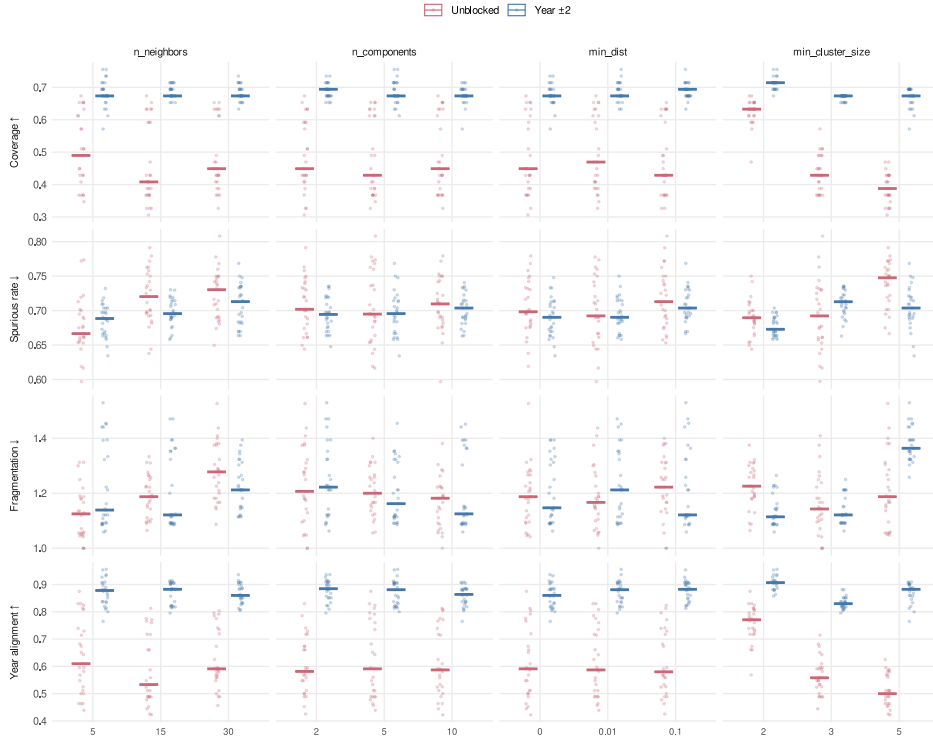
Figure 15: **Embedding-geometry probe in UMAP space.** F16 vs FP32, the same model precisions contrasted in Table 9 at no-UMAP. Columns facet the two probe metrics; rows facet embedding precision (F16 = local Ollama community port, FP32 = DeepInfra-hosted reference). The F16 panels (top row) reproduce the earlier UMAP read; the FP32 panels (bottom row) add the missing precision contrast in UMAP space. Each bar counts UMAP cells (out of 27) whose probe value falls in the bin; fill marks eval slice (Full pool = Tier 1 + 2 unambiguous, Tier 1 = verbatim-passage matches only). Dashed lines are the F16/FP32 floors carried over from Table 9: 0.40 for top-1 NN; 0 for separation (Euclidean — the 0.05 cosine floor at Table 9 does not translate). Identical UMAP grid, seed (20260526), and probe logic across the four panels; the only thing that varies between F16 and FP32 rows is the input embedding matrix.



Hyperparameter marginal scan (UMAP grid)

The 162-variant UMAP grid sweeps four knobs at once — three UMAP hyperparameters (`n_neighbors`, `n_components`, `min_dist`) crossed with `min_cluster_size`, all under `{unblocked, year ±2}`. Figure 16 and Figure 17 scan that grid one knob at a time: each column is a hyperparameter, the x-axis steps through its three swept values, and every point is one of the variants holding that value while the other three knobs vary. Because three knobs vary at every x-position, each column is a **marginal** view — the spread of points at a given x is the variation induced by the other three knobs, and the crossbar is the median over them. Rows are outcomes (as in Method 1), colour is year-blocking, and the two gates split across sub-panels. A knob that mattered would tilt its column’s medians; a flat column means the front does not move along that knob — the empirical read developed in §“Reading the UMAP results” below.

(a) Keyword gate



(b) JW-min ≤ 0.30 gate

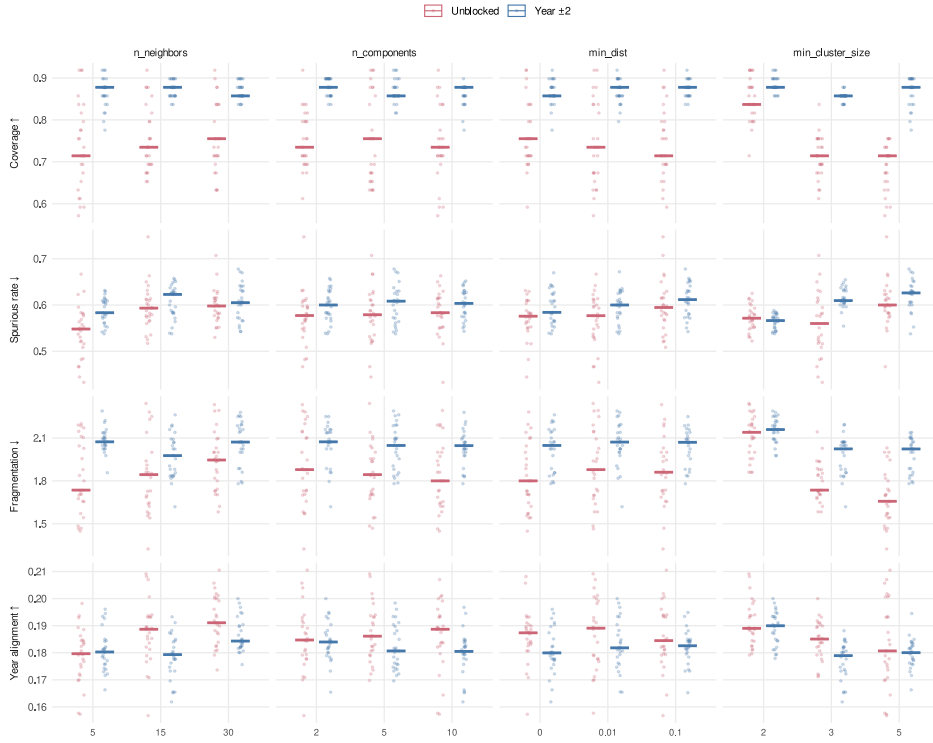


Figure 16: **UMAP headline metrics — marginal scan.** Columns: the four swept knobs (`n_neighbors`, `n_components`, `min_dist`, `min_cluster_size`). Rows: the four headline metrics, $\uparrow$ / $\downarrow$ in the strip label. x steps through each knob's three values; each point is one of the 162 variants at that value (the other three knobs vary), with a median crossbar per (value $\times$ blocking). Colour: blocking. Gates split across sub-panels. A flat column = that knob does not move the metric. Y-axes free per metric row; x free per knob column.

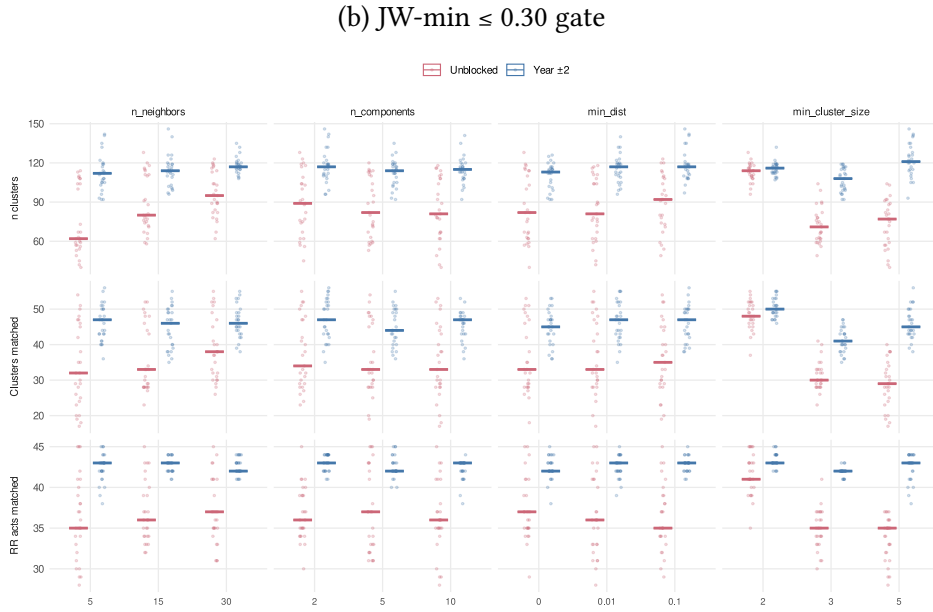
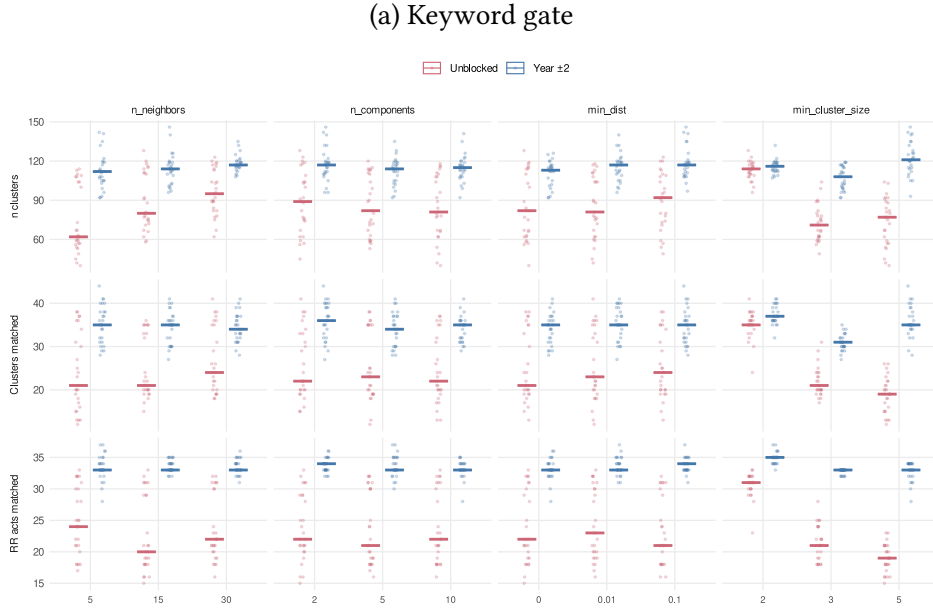


Figure 17: **UMAP match counts — marginal scan.** Same layout as Figure 16 but plotting raw counts. Rows: total clusters emitted, clusters carrying an RR match, RR acts covered (out of 49). `n_clusters` is gate-invariant so its row repeats across the two sub-panels. Points are exact counts (no bootstrap CI); the crossbar is the median over the marginalized knobs.

Reading the UMAP results

The headline reading is empirical: across the 162 UMAP variants in `c0_hdbscan_umap_rr_metrics`, **no UMAP variant strictly dominates the no-UMAP HDBSCAN baseline on the coverage/spurious front under either gate**. The shortlist tables above pick the best UMAP cell per (`min_cluster_size` × year-blocking) sub-cell by the asymmetric-cost rule (lowest spurious, then highest coverage); the picks land at comparable spurious rates and comparable

coverage to the corresponding no-UMAP rows in Table 10 and Table 12. UMAP-PF (the precision-friendly UMAP winner overall) does cut the singleton share in the cluster-count distribution from 38% (no-UMAP baseline `hdb_m2_yw2`) to 14%, but most of the singletons it removes are noise points that wouldn't have matched any RR act anyway. Two independent diagnostics explain why the front doesn't move.

First, **UMAP improves the aggregate embedding geometry but not the neighbour identity HDBSCAN and the name gates actually use**. Figure 15 shows within-vs-between separation moves dramatically — F16 raw 0.005 $\rightarrow$ UMAP median 0.379, a $\approx 70\times$ lift, with positive separation in 27/27 F16 cells and 26/27 FP32 cells. Top-1 NN same-act rate barely moves: F16 raw 0.261 $\rightarrow$ UMAP cells span [0.18, 0.31], and zero of 108 probe cells (54 F16 + 54 FP32 across both eval slices) clear the 0.40 floor. The F16 $\leftrightarrow$ FP32 contrast on top-1 NN is flat, which resolves the verdict left open by the no-UMAP F16/FP32 probe in §“Embedding-geometry probe”: the bottleneck is the embedding model, not F16 quantization. UMAP cleans aggregate geometry without changing the literal nearest neighbour, and the literal NN is what HDBSCAN's density-based clustering and the RR-aligned name gates both depend on.

Second, **the upstream gold-pool ceiling documented in §“Why we pivoted away from pairwise P/R” still binds the RR-aligned eval, just differently**. The new framework removes the chunk-tier `act_name` aggregation noise that capped pairwise recall against `c0_eval_gold_pairs`, but coverage against R&R's 49-act list is still capped by whether HDBSCAN finds ≥ 1 surface form per act that carries enough of the canonical name to satisfy the gates. Tiny clusters that fragment off the abbreviated forms (ERTA, P.L. 97-34) without merging back to the canonical fail both gates regardless of UMAP arrangement. The earlier UMAP draft's third hypothesis (that `n_components` would be the load-bearing UMAP knob) remains empirically wrong on the RR-aligned eval too: no UMAP knob moves the front meaningfully because the front is not where UMAP's lever points. BERTopic-style UMAP-before-density-clustering is therefore not the load-bearing C0 architecture.

Implications for Method 4. The same diagnosis predicts the hybrid LLM pairwise judge will break through *some* of the gap but not all of it. Genuine surface variants (ERTA $\leftrightarrow$ Economic Recovery Tax Act of 1981 $\leftrightarrow$ P.L. 97-34) that HDBSCAN scattered by 1024-D cosine-distance concentration will resolve once the judge sees the candidate pairs — top-k embedding retrieval should surface them as candidates even at F16 (top-1 may miss but top-5/10 will catch). The judge cannot manufacture canonical names from scratch, though, so methods that emit canonicals (Method 5) will look better on the keyword gate than methods that only verify pairs (Method 4); the keyword-vs-JW gap in the §Synthesis ladder will surface that distinction.

Method 4 — Hybrid (embedding nearest-neighbours + LLM pairwise judge)

⚠ Phase B pending

The hybrid pipeline (embedding top-k → Haiku YES/NO verifier → union-find) lands once Phase B is built. Expected to dominate the cost/quality frontier: cheap blocker + targeted LLM verification only on the candidate pairs that survive embedding similarity.

Method 5 — LLM canonical clustering

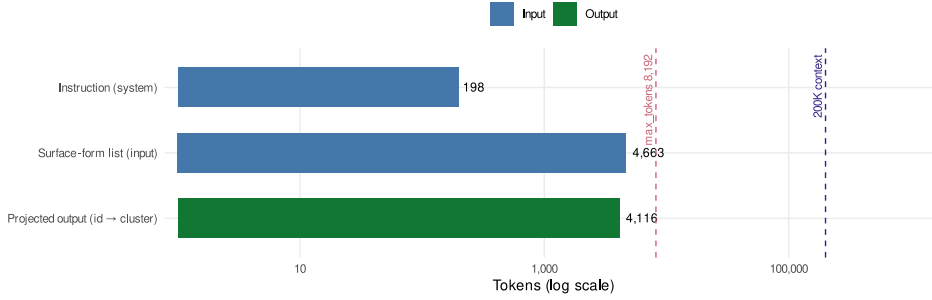
Method 5 is the semantic bridge Methods 1–3 cannot build on their own (§“What this means for Method 1”). It is an **unsupervised** clustering method: Haiku is shown the full pool of distinct rank-1 surface forms — each with a representative year — and returns, for every name, a cluster key (`surface_id` → `cluster_id`). It never sees R&R’s act list; the gold labels enter only in evaluation, through the same `match_clusters_to_rr_acts()` + `evaluate_rr_matches_grid()` gates Methods 1–3 are scored under. The model emits a grouping of **ids**, never strings, so there is nothing to fuzzy-match on the output side and the members the gates score are the original C1 surface forms — M5 is graded apples-to-apples with the lexical and embedding methods, with no free credit for re-emitting canonical names. The bespoke clustering instruction lives in `c0_canonicalize.yml`; canonical-name selection and the reshape to the standard cluster table are deterministic R in `R/c0_aggregator.R` (`run_m5_llm_clusters()`).

Iteration 1 is the barebones cell of the design: **Haiku, single-shot, names + representative year, five fixed-seed input shuffles**. The five seeds are the order-sensitivity probe (the M5 analog of C1/C2’s Test IV order-invariance test): each seed reorders the input list, becomes its own `variant_id`, and is scored independently, so seed-to-seed metric spread and partition agreement quantify how much the answer rides on input order.

Context-window headroom

Before spending any tokens, Figure 18 confirms the single-shot call clears both the context window and our output budget. The id-only output schema keeps the response compact — the model echoes one short `{id, cluster}` record per name rather than the full surface string — so even though it returns one record per surface form, the projected output stays well under `max_tokens`. Output is estimated at ~14 tokens per record (the JSON structure costs roughly twice the chars/4 heuristic, a correction the iteration-1 pilot surfaced when a 4,096-token budget truncated the response).

Figure 18: **M5 token budget**. Approximate tokens for the system instruction, the surface-form input list (chars/4), and the projected `id→cluster` output (~14 tokens per pretty-printed JSON record), against our `max_tokens` output budget (8,192) and the Haiku/Sonnet 200K context window (both dashed). Log x-axis. Input sits at ~2% of the window; output sits below the configured budget — single-shot is feasible at iteration 1’s pool size.



Partition integrity

Table 16 is the first reliability gate: did the model return a clean partition of all 294 input ids on each seed? Missing ids are filled as singleton clusters (recoverable, under-merge), extra/hallucinated ids are dropped. Non-zero `Missing` or `Extra` columns, or a non-`end_turn` stop reason, flag output that the model truncated or drifted on.

Table 16: **M5 partition integrity, per seed**. `Missing` → `singleton` ids were not returned and are filled as singletons; `Extra` (`dropped`) ids were returned but not in the input. A clean run shows 0 / 0 and `end_turn`.

Seed	Names expected	Returned	Missing → single- ton	Extra (dropped)	Parsed	Stop reason
1	294	294	0	0	TRUE	end_turn
2	294	294	0	0	TRUE	end_turn
3	294	294	0	0	TRUE	end_turn
4	294	294	0	0	TRUE	end_turn
5	294	294	0	0	TRUE	end_turn

Order sensitivity

Table 17 reports the adjusted Rand index between every pair of the five seed partitions. $ARI = 1$ means input order has no effect on the clustering; lower values mean early-listed names anchor clusters the later order would have drawn differently. This is the headline robustness number for a single-shot LLM clusterer.

Table 17: **M5 order sensitivity.** Pairwise adjusted Rand index across the five shuffle-seed partitions of the shared name pool. 1.0 = order-invariant.

Seed variants	Seed pairs	Mean ARI	Min ARI	Max ARI
5	10	0.752	0.654	0.839

RR-aligned metrics

The same two-gate ladder used for Methods 1–3, one row per seed. Coverage / spurious / fragmentation / year-alignment carry 95% bootstrap CIs (act-resample for the act-indexed metrics, cluster-resample for spurious). The keyword gate is M5’s honest headline (immune to canonical re-emission); the JW gate is shown for parity with the other methods’ tables. Read the seed-to-seed spread as the order-sensitivity envelope on each metric.

Table 18: **Method 5, keyword gate.** One row per shuffle seed. Coverage $\uparrow$, Spurious $\downarrow$, Fragmentation $\downarrow$, Year alignment $\uparrow$.

variant_id	Coverage	Spurious	Fragmentation	Year alignment	n clusters	ClusRR acts matched	ters matched
m5_haiku_namesyear51	0.751 [0.633, 0.857]	0.785 [0.727, 0.839]	1.189 [1.077, 1.292]	0.978 [0.958, 1.000]	205	44	37
m5_haiku_namesyear52	0.752 [0.633, 0.857]	0.801 [0.741, 0.856]	1.081 [1.000, 1.143]	0.976 [0.955, 1.000]	201	40	37
m5_haiku_namesyear53	0.753 [0.633, 0.857]	0.796 [0.741, 0.851]	1.108 [1.000, 1.190]	0.976 [0.957, 1.000]	201	41	37
m5_haiku_namesyear54	0.754 [0.633, 0.857]	0.789 [0.732, 0.847]	1.189 [1.045, 1.304]	0.978 [0.958, 1.000]	209	44	37
m5_haiku_namesyear55	0.755 [0.633, 0.857]	0.797 [0.744, 0.850]	1.135 [1.000, 1.227]	0.977 [0.955, 1.000]	207	42	37

Table 19: **Method 5, JW gate.** Same rows, JW-min ≤ 0.30 name gate.

variant_id	Coverage	Spurious	Fragmentation	Year alignment	n clusters	Clusters matched	RR acts matched
m5_haiku_namesyear18	0.918 [0.837, 0.980]	0.698 [0.634, 0.756]	2.333 [2.028, 2.630]	0.185 [0.161, 0.208]	205	62	45
m5_haiku_namesyear18	0.918 [0.837, 0.980]	0.701 [0.637, 0.761]	2.311 [1.966, 2.643]	0.185 [0.160, 0.210]	201	60	45
m5_haiku_namesyear18	0.918 [0.837, 0.980]	0.697 [0.637, 0.761]	2.333 [2.000, 2.655]	0.186 [0.160, 0.212]	201	61	45
m5_haiku_namesyear18	0.918 [0.837, 0.980]	0.699 [0.636, 0.761]	2.356 [2.034, 2.655]	0.185 [0.162, 0.208]	209	63	45
m5_haiku_namesyear18	0.918 [0.837, 0.980]	0.705 [0.643, 0.768]	2.333 [2.034, 2.640]	0.186 [0.162, 0.210]	207	61	45

Frequency as an unsupervised RR signal

In deployment there is no ground truth, so we need a corpus-internal property that ranks which clusters merit expert review. The hypothesis: R&R acts are *significant* legislated fiscal changes, and significance tracks how often an act is discussed, so RR-matched clusters should sit higher in the per-cluster frequency distribution than the spurious tail. The two diagnostics below test that separability on the US corpus (where RR labels exist) for a single seed (`s1`); coverage is seed-stable so the seed choice does not move the marks. This is a qualitative separability look, not a fitted threshold — what transfers to Malaysia is the *signal*, not a cutoff.

Figure 19: **M5 cluster frequency by RR match.** Distinct US chunks referencing each M5 cluster (seed s1, log x), stacked by Romer–Romer match gate; the rug marks individual RR-matched clusters. Unmatched clusters pile at one to two chunks while RR-matched clusters dominate the high-frequency tail.

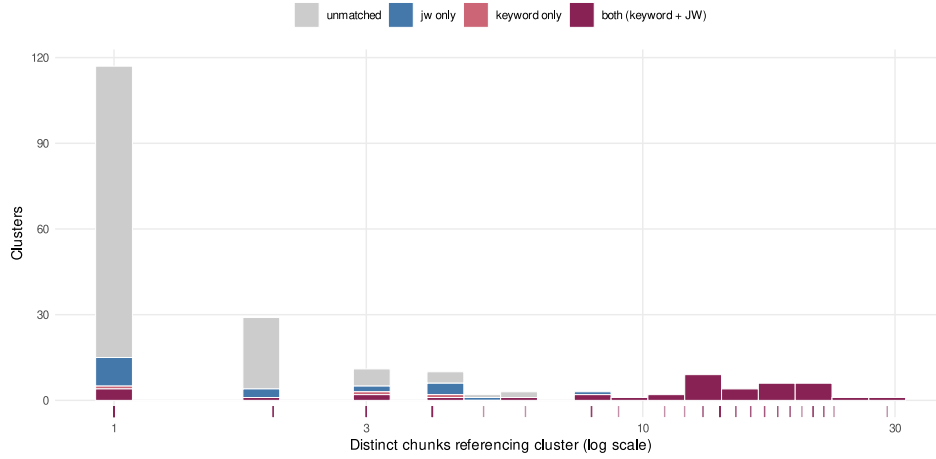


Figure 20: **Canonicalized act-names by frequency of appearance in US corpus.** Matched and unmatched by the keyword gate.

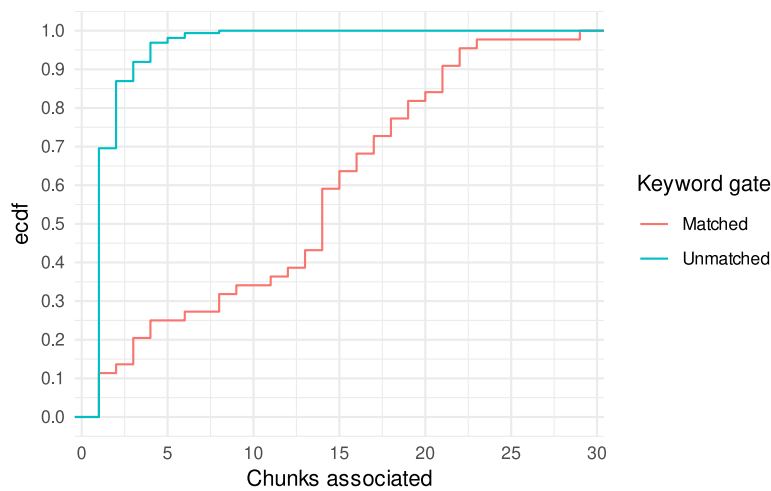


Figure 20 shows that up to 85% of unmatched act-names—acts that were surfaced by C0 but are not found in Romer and Romer (n.d.) ‘s set of fiscal shocks—are only found in at most two chunks inside the whole C1 test sample. At the same time, fewer than 10% of the matched acts are found in 2 or fewer chunks. This is to say that, if we chose to chop the “tree” of acts surfaced by C0 by their occurrence in the chunks, we would only lose 10% of matched acts and only keep 15% of the unmatched acts.

Figure 21: **Trade-off of frequency based identification.** Cumulative errors (as un/matched by keyword gate) across frequency thresholds. Canonicalized act names are arranged by their frequency in chunks

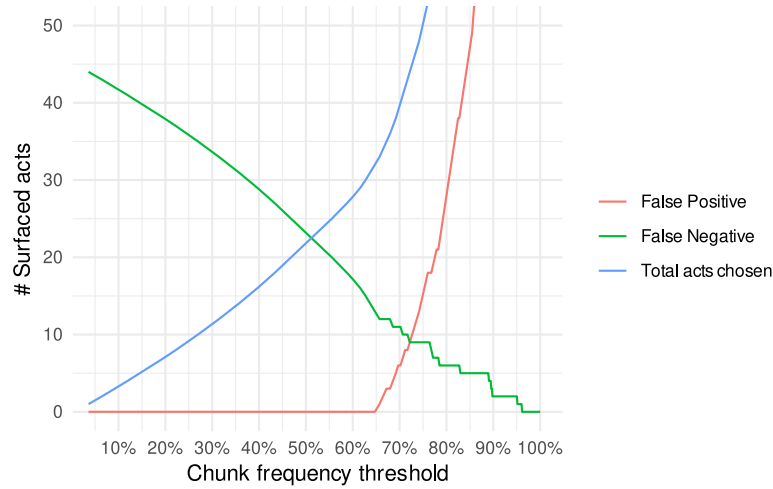
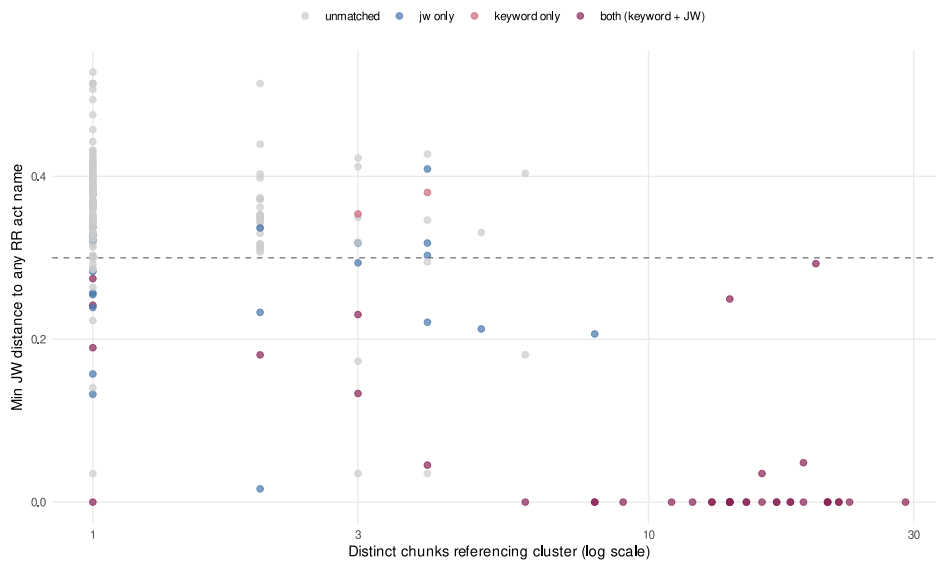


Figure 21 shows the trade-off curves in terms of false positive and false negative canonicalized act names. Keeping the 70% most frequently used canonical act names will produce 10 false negatives and 6 false positives.

Figure 22: **M5 frequency vs name distance.** Each M5 cluster (seed s1): corpus frequency (log x) against the smallest Jaro–Winkler distance from its canonical name to any of the 49 R&R act names. The dashed line is the 0.30 JW name gate; the keyword gate (term + year) shows by colour, not a line. Distance is name-only and year-agnostic, so a low-distance point can still be year-wrong — hence the trustworthy keyword gate is encoded separately. RR-matched clusters form an L: low name distance and high frequency.



Both diagnostics support the hypothesis. The unmatched mass concentrates at one to two chunks, RR-matched clusters carry the high-frequency tail, and in the

scatter the RR-matched clusters fall to low name distance and high frequency while the spurious tail sits at single-chunk frequency and large name distance. Frequency alone will not isolate RR-grade acts — RR clusters do appear in the single-chunk bin and some spurious clusters reach moderate frequency — but as a triage ranking for expert review it clearly demotes the spurious tail. Quantifying the separation (an AUC or precision–recall on frequency) and a persistence axis (distinct years and documents) are the natural next steps.

Method comparison ladder

Performance frontier

The ladder tables below collapse each method to a single “best” variant (lowest spurious, then highest coverage). Figure 23 shows the full picture that pick is drawn from: every one of the 183 implemented variants as a point in coverage–spurious space, per gate, with the pooled efficient frontier (the non-dominated envelope — maximum coverage achievable at each spurious level) drawn over them. This is the honest “how well can the methods perform” answer for objective (i): a frontier, not a point.

Two structural facts read straight off it. Within JW and within no-UMAP HDBSCAN there is **no** coverage–spurious tradeoff — one variant dominates all others of its method on both axes, so each contributes a single frontier point; the only genuine tradeoff *curve* is the UMAP grid. And the three methods occupy different regions: JW reaches the highest coverage (≈ 0.84) but only at spurious ≥ 0.95 ; HDBSCAN + UMAP owns the low-spurious end (down to ≈ 0.60) at comparable coverage. The deployment pick in Table 20 is just the lowest-spurious *corner* of this frontier — a different cost ratio (false merges per missed act) would pick a different corner.

Figure 23: **Coverage–spurious performance frontier.** All 183 implemented variants (10 JW + 6 HDBSCAN + 162 HDBSCAN/UMAP + 5 LLM M5 seeds) in coverage ($\uparrow$ better) $\times$ spurious-rate ($\leftarrow$ better) space, per name gate. Colour: method; shape: year-blocking. The grey line is the pooled efficient frontier (non-dominated envelope across all methods); enlarged points sit on it. JW and no-UMAP HDBSCAN each contribute a single frontier point (one variant dominates the rest of its method); the UMAP grid supplies the tradeoff curve at low spurious; the five M5 seeds show the LLM’s order-sensitivity scatter. Point estimates only (CIs omitted for legibility; see the ladder tables).

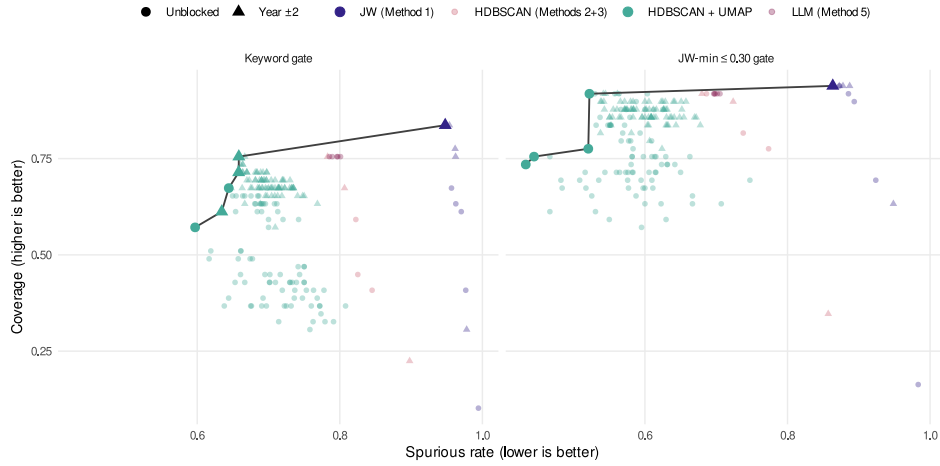


Figure 24 decomposes the same variants into the raw count geometry behind the rates: x is how many emitted clusters carry an RR match, y is how many of the 49 RR acts are covered, and point area is the total cluster budget ($n_{clusters}$) — the denominator that drives the spurious rate. The dashed line is $y = x$ (one matched cluster per covered act, no fragmentation among matches); points to its right are fragmented (several clusters matching the same act). JW sits far right with the largest points — it covers many acts but emits a vast cluster budget and fragments heavily; HDBSCAN + UMAP clusters near the line with small budgets, the count-space signature of its low spurious rate.

Figure 24: **Match-count geometry**. Same 183 variants, per gate. x: clusters carrying an RR match. y: RR acts covered (of 49). Point area: total clusters emitted (`n_clusters`, the spurious-rate denominator). Colour: method; shape: year-blocking. Dashed line is $y = x$ (no fragmentation among matched clusters); points to its right fragment one act across several clusters. Large points far right (JW) = big budget + heavy fragmentation; small points near the line (UMAP) = lean budget.

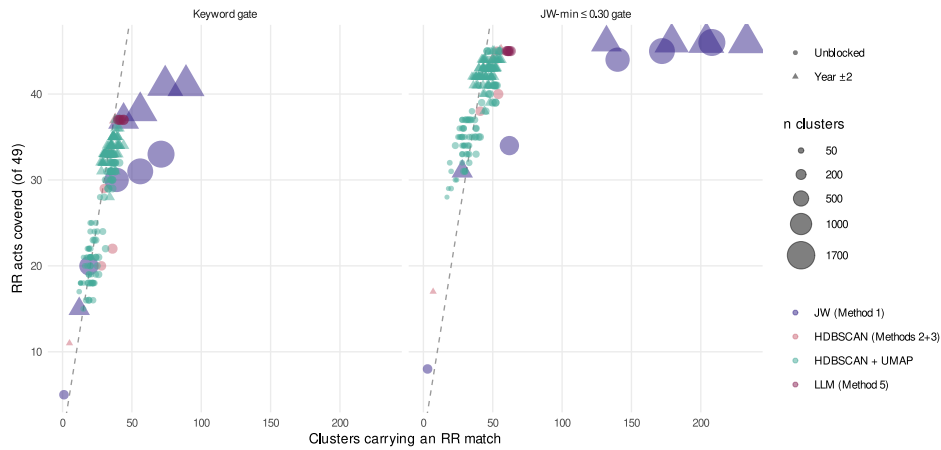


Table 20: **Method comparison, keyword gate**. Best variant per method under the asymmetric-cost rule (lowest spurious_rate, then highest coverage). Methods 4 + 5 are Phase B-pending.

	Method	Vari- ant	Cov- erage	Spu- rious	Frag- men- tation	Year align- ment	n clus- ters	Clus- ters matched	RR matched
	JW	jw_t0.10_y0.837 (best vari- ant)	0.837 [0.714, 0.939]	0.448 [0.226, 0.958]	2.268 [1.958, 2.583]	0.836 [0.845, 0.929]	1707	89	41
	HDBSCAN	hdb_m2_y0.735 (best vari- ant)	0.735 [0.603, 0.857]	0.383 [0.724, 0.846]	1.108 [1.000, 1.182]	0.976 [0.955, 1.000]	175	38	37
hdb_umap	HDBSCAN	hdb_m0_md0.050_y0.305 + UMAP (best vari- ant)	0.571 [0.305, 0.714]	0.595 [0.466, 0.716]	1.461 [1.000, 1.133]	0.814 [0.609, 0.815]	67	27	28
	LLM	llm_method_names5 (best seed)	0.755 [0.551, 0.857]	0.385 [0.721, 0.839]	1.789 [1.000, 1.292]	0.978 [0.958, 1.000]	205	44	37

Table 21: **Method comparison, JW gate.** Same picker, JW-min ≤ 0.30 gate. The keyword-vs-JW gap diagnoses recovery-by-near-canonical-match; M5’s members are the original C1 surface forms (it never emits new strings), so it gets no extra JW credit Methods 1-3 don’t.

Method	Vari- ant	Cov- erage	Spu- rious	Frag- men- tation	Year align- ment	n clus- ters matched	Clus- ters matched	RR
JW	jw_t0.10_0v939	[0.873, 1.000]	[0.864, 0.880]	[0.843, 8.429]	[6.584, 79.0201]	1707	233	46
(best vari- ant)								
HDBSCAN	hdb_m2_y0v218	[0.873, 0.980]	[0.864, 0.749]	[0.843, 2.519]	[6.584, 1.90388]	175	56	45
(best vari- ant)								
hdb_umap	hdb_umap_m0md00735	[0.873, 0.857]	[0.864, 0.552]	[0.843, 2.304]	[6.584, 1.72177]	67	38	36
(best vari- ant)								
LLM	llm_method_name0s918	[0.873, 0.980]	[0.864, 0.761]	[0.843, 2.655]	[6.584, 2.00086]	201	61	45
(best seed)								

Four rows now — JW, no-UMAP HDBSCAN, UMAP+HDBSCAN, and LLM (Method 5, best seed) — picked by the asymmetric-cost rule within each method. Reading across the rows: if UMAP+HDBSCAN dominates the no-UMAP row on coverage with comparable-or-lower spurious, BERTopic-style dim-reduction is the load-bearing improvement. If JW wins on spurious but trails on coverage (the expected pattern given the within-act JW geometry), JW remains a precision-friendly blocker candidate for Method 4 rather than a deployable method on its own. Method 5 is the test of whether an LLM’s semantic grouping clears the coverage/spurious frontier the lexical and embedding methods are stuck behind; its row here is the best of five shuffle seeds, so read it together with the seed spread in Table 18 (the best-seed pick is optimistic). Method 4 will extend this table once it lands.

Out-of-distribution: Malaysia EN/BM

⚠ Phase B pending

Once Phase B is built, the best variant from each method is applied to the Malaysia EN+BM measure pool (`malay_er_cl_measures`); we report the fraction of `malay_er_curated_matches` EN↔BM pairs that land in the same predicted cluster. Methods that rely on surface similarity (JW) are expected to fail here because the language families are different; multilingual embedding methods should hold up.

Cost diagnostics

Table 22: **Per-method cost.** Methods 1–3 are zero-cost (pure-R or local Ollama). Method 5 is one Haiku single-shot call per shuffle seed (5 seeds); token columns are the chars/4 estimate $\times$ 5, dollar costs are upper bounds pending `logs/api_calls.csv`. Method 4 still pending.

Method	Tokens (input)	Tokens (output)	Pilot cost (USD)	Projected SEA cost (USD)	Notes
Method 1: JW single- linkage	0	0	0.00	0.00	Pure-R, no API calls
Method 2: E5-mul- tilingual + HDB- SCAN	$\approx$ 4K	0	0.00	0.00	Local Ollama (jeffh/int- float-mul- tilingual- e5-large-instruct:f16); embed- dings cached in <code>c0_us_embeddings</code>
Method 3: E5-mul- tilingual + HDB- SCAN (year- blocked)	$\approx$ 4K	0	0.00	0.00	Reuses Method 2 embed- dings; year- blocking applied at the dis- tance-ma- trix step
Method 5: Haiku sin- gle-shot LLM clus- tering	$\approx$ 25K (5 $\times$ $\approx$ 5K)	$\approx$ 20K (5 $\times$ $\approx$ 4K)	<0.10	<0.50	id-only output keeps tokens small; 5 seeds for the order- sensitivity probe. Ex- act cost in <code>logs/api_calls.csv</code>

⚠ Method 4 still pending

Phase B cost diagnostics for Method 4 (Haiku pairwise-judge) will populate from `logs/api_calls.csv` once that target runs.

Synthesis

i Method 4 still pending

A final method + parameter recommendation waits for Method 4 to be evaluated. Methods 1–3 (JW, HDBSCAN unblocked, HDBSCAN year-blocked) and Method 5 (LLM canonical clustering) are now wired against the RR-aligned eval; Method 4 (hybrid embedding-NN + LLM pairwise judge) remains to land.

Until then, the shortlist below applies the precision-friendly rule from §Framing to each finished method in isolation: among variants with the lowest spurious rate, pick the one with the highest coverage. The load-bearing comparison is now Method 5's best seed against the JW and HDBSCAN picks — whether the LLM's semantic grouping clears the coverage/spurious frontier the lexical and embedding methods are stuck behind.

Reading the JW results in isolation

Table 23: **JW shortlist, keyword gate.** Under the asymmetric-cost rule (sort: spurious asc, coverage desc). The top row is the most precision-friendly JW variant; the Coverage column says whether that precision came at an unacceptable recall hit.

variant_id	Spurious	Coverage	n clusters	Clusters matched	RR acts matched
jw_t0.10_yw2	0.948 [0.937, 0.958]	0.837 [0.714, 0.939]	1707	89	41
jw_t0.15_yw2	0.954 [0.944, 0.964]	0.837 [0.714, 0.939]	1602	74	41
jw_t0.10_unblocked	0.957 [0.947, 0.966]	0.673 [0.531, 0.796]	1638	71	33
jw_t0.20_yw2	0.962 [0.952, 0.971]	0.776 [0.653, 0.878]	1465	56	38
jw_t0.25_yw2	0.962 [0.950, 0.974]	0.755 [0.633, 0.857]	1170	44	37

Table 24: **JW shortlist, JW gate.** Same JW shortlist on the JW gate. Coverage typically lifts on the relaxed gate because surface variants share lexical content; the keyword-vs-JW coverage gap is the recovery-by-near-canonical bonus.

variant_id	Spurious	Coverage	n clusters	Clusters matched	RR matched	acts matched
jw_t0.10_yw2	0.864 [0.848, 0.880]	0.939 [0.877, 1.000]	1707	233	46	
jw_t0.15_yw2	0.873 [0.857, 0.888]	0.939 [0.877, 1.000]	1602	204	46	
jw_t0.10_unblocked	0.873 [0.857, 0.889]	0.939 [0.877, 1.000]	1638	208	46	
jw_t0.20_yw2	0.878 [0.861, 0.894]	0.939 [0.877, 1.000]	1465	179	46	
jw_t0.15_unblocked	0.885 [0.870, 0.901]	0.918 [0.837, 0.980]	1502	172	45	

This shortlist is sorted by spurious rate ascending — the column the C0 user should weight most under the asymmetric-cost framing. The top row is the most precision-friendly JW variant; the Coverage column says whether the spurious-rate win came at acceptable coverage cost.

Reading the HDBSCAN results in isolation

Table 25: **HDBSCAN shortlist, keyword gate.** No-UMAP and UMAP-reduced combined, under the asymmetric-cost rule. **Method** marks which family each row comes from.

	Method	variant_id	Spuri- ous	Cover- age	n clus- ters	Clusters matched	RR acts matched
hdb_umap	HDB	hdb5_nc10_md0.010_m397	unblocked	0.571	67	27	28
	SCAN +		[0.463,	[0.429,			
	UMAP		0.716]	0.714]			
hdb_umap	HDB	hdb5_nc5_md0.010_m317	unblocked	0.490	60	23	24
	SCAN +		[0.483,	[0.367,			
	UMAP		0.733]	0.633]			
hdb_umap	HDB	hdb5_nc5_md0.000_m319	unblocked	0.510	63	24	25
	SCAN +		[0.492,	[0.367,			
	UMAP		0.731]	0.653]			
hdb_umap	HDB	hdb5_nc5_md0.000_m335	unblocked	0.612	93	34	30
	SCAN +		[0.538,	[0.469,			
	UMAP		0.731]	0.735]			
hdb_umap	HDB	hdb15_nc5_md0.000_m338	unblocked	0.367	58	21	18
	SCAN +		[0.517,	[0.245,			
	UMAP		0.759]	0.510]			
hdb_umap	HDB	hdb15_nc5_md0.010_m344	unblocked	0.388	59	21	19
	SCAN +		[0.525,	[0.245,			
	UMAP		0.763]	0.531]			
	HDB-SCAN	hdb_m2_yw2	0.783	0.755	175	38	37
	SCAN		[0.726,	[0.633,			
			0.846]	0.857]			
	HDB-SCAN	hdb_m3_yw2	0.807	0.673	181	35	33
	SCAN		[0.746,	[0.531,			
			0.862]	0.796]			

Table 26: **HDBSCAN shortlist, JW gate.** Same HDBSCAN shortlist on the JW gate.

	Method	variant_id	Spuri- ous	Cover- age	n	clus- ters	Clusters matched	RR acts matched
hdb_umap	HDB- hdb_m5_nc10_md0.010_unblocked	0.333	0.333	0.735	67	38	36	
	SCAN +		[0.313,	[0.612,				
	UMAP		0.552]	0.857]				
hdb_umap	HDB- hdb_m5_nc5_md0.000_unblocked	0.344	0.344	0.755	63	35	37	
	SCAN +		[0.317,	[0.633,				
	UMAP		0.556]	0.878]				
hdb_umap	HDB- hdb_m5_nc5_md0.010_unblocked	0.367	0.367	0.755	60	32	37	
	SCAN +		[0.333,	[0.633,				
	UMAP		0.584]	0.878]				
hdb_umap	HDB- hdb_m5_nc2_md0.010_unblocked	0.367	0.367	0.712	45	24	30	
	SCAN +		[0.311,	[0.490,				
	UMAP		0.600]	0.735]				
hdb_umap	HDB- hdb_m5_nc2_md0.000_unblocked	0.382	0.382	0.794	56	29	34	
	SCAN +		[0.357,	[0.551,				
	UMAP		0.625]	0.816]				
hdb_umap	HDB- hdb_m5_nc2_md0.010_unblocked	0.384	0.384	0.773	62	32	33	
	SCAN +		[0.355,	[0.531,				
	UMAP		0.613]	0.796]				
HDB- hdb_m2_yw2	HDB- hdb_m2_unblocked	0.680	0.680	0.918	175	56	45	
	SCAN		[0.606,	[0.837,				
			0.749]	0.980]				
HDB- hdb_m2_unblocked	HDB- hdb_m2_unblocked	0.686	0.686	0.918	169	53	45	
	SCAN		[0.621,	[0.837,				
			0.757]	0.980]				

Same sort rule as the JW shortlist. Reading across the three families (JW above, no-UMAP HDBSCAN, UMAP+HDBSCAN): the deployment pick under each gate is the top row. If a UMAP+HDBSCAN variant displaces no-UMAP HDBSCAN at the top, deploy with UMAP for Malaysia. If JW dominates spurious but trails coverage, JW is the natural blocker candidate for Method 4 – the hybrid pipeline reconciles the spurious-vs-coverage trade by routing JW’s precision-friendly candidate pairs through a semantic verifier.

Romer, Christina D., and David H. Romer. n.d. *A Narrative Analysis of Postwar Tax Changes*.